

Calibrating the Sorites Paradox:
Interactive Threshold Tuning and
Cost-Sensitive Evaluation in
Credit Card Fraud Detection

Dissertation

COMP702 — M.Sc. Project (2025/26)

Submitted by

Nidhi Kumari

(Student number: 201960034)

under the supervision of

Dr Vitaliy Kurlin

SCHOOL OF COMPUTER SCIENCE AND INFORMATICS
UNIVERSITY OF LIVERPOOL

Dedication

For my family, whose steady support made a full-time M.Sc. alongside part-time work possible, and for anyone who has ever had to draw a hard line through a soft, uncertain question.

Student Declaration

I confirm that I have read and understood the University's Academic Integrity Policy.

I confirm that I have acted honestly, ethically and professionally in conduct leading to assessment for the programme of study.

I confirm that I have not copied material from another source nor committed plagiarism nor fabricated data when completing the attached piece of work. I confirm that I have not previously presented the work or part thereof for assessment for another University of Liverpool module. I confirm that I have not copied material from another source, nor colluded with any other student in the preparation and production of this work.

I confirm that I have not incorporated into this assignment material that has been submitted by me or any other person in support of a successful application for a degree of this or any other university or degree-awarding body.

SIGNATURE

DATE

[Insert submission date]

Acknowledgements

I would like to thank my supervisor, Dr Vitaliy Kurlin, for his guidance throughout this project, in particular for pushing me to sharpen the philosophical framing of the work, to justify design choices in the proposal rather than merely describe them, and to keep the mathematical exposition accessible without losing precision. His feedback measurably improved both the proposal and the system built from it.

I am grateful to my classmates on the M.Sc. in Advanced Data Science and Artificial Intelligence for many useful discussions about threshold-based classification and about surviving a taught M.Sc. year while working part-time.

Finally, I would like to thank my family and friends for their patience and encouragement during a demanding year that combined full-time study with part-time work.

Calibrating the Sorites Paradox: Interactive Threshold Tuning and Cost-Sensitive Evaluation in Credit Card Fraud Detection

Abstract

When a system has to decide whether something is suspicious enough to act on, where exactly should the line be drawn? This project argues that the line is not a fact waiting to be discovered, but a value judgement about which mistake matters more, and that it should move whenever that judgement changes rather than being fixed once and left alone. The idea is framed around an old philosophical puzzle, the Sorites Paradox: a heap of sand does not stop being a heap after just one grain is removed, and yet removing enough grains clearly does leave a non-heap, so no single grain ever marks the true boundary. Deciding whether a transaction is fraudulent enough to block runs into a quiet version of exactly the same problem.

The practical outcome is Threshold Lab, an interactive tool built around a bank fraud scenario. Rather than using a fixed, one-size-fits-all cut-off, it asks what the two kinds of mistake actually cost — wrongly blocking a genuine customer, versus missing a real fraud — and works out, from those costs, exactly where the line should sit. It compares a few different ways of building and checking the underlying prediction model, and lets a user change their assumptions about cost on the fly and immediately see how the decision, and its financial consequences, change.

This project's original idea is the *Contested Zone*: rather than only flagging cases the model itself is unsure about, it identifies the transactions whose outcome would flip depending on which of two equally reasonable views about cost is taken. A modest but meaningful share of transactions, representing a real amount of money, turn out to fall into this zone of genuine disagreement between reasonable people, rather than into any error the model is making. The dissertation also reports, honestly rather than selectively, a result that did not go as expected: the simplest model tried performed better than the more sophisticated ones on this particular dataset, and the write-up discusses why. It closes with a candid evaluation of the finished system against its original goals, what a proper study involving real people would still need to add, and how several classical philosophical responses to the Sorites Paradox each point towards a different way such a system could be built.

Statement of Ethical Compliance

This project did not collect any primary data from human participants: no interviews, questionnaires, usability studies or other activities involving people were conducted, and Chapter 3 explains why, together with what a future iteration involving real users would need to add. The only data source used is the ULB Credit Card Fraud Detection dataset (Dal Pozzolo et al., 2015), a publicly available, third-party, pre-anonymised dataset of European cardholders' transactions in which every identifying feature has already been replaced by the original authors with Principal Component Analysis (PCA) components; no transaction in the dataset, real or synthetic, can be traced back to an individual. Where the authentic dataset is unavailable, the accompanying software falls back to a purely synthetic data generator that produces no real personal or transactional data whatsoever. No participant consent form is therefore included in the appendices, as none was required.

Under the module's ethical guidance and categorisation scheme, this project is assessed as falling into the lowest-risk category — [*Category: to be confirmed by the student against the current COMP702 ethical guidance page, e.g. A0*] — corresponding to a project that involves no primary collection of personal data from human participants and uses only existing, publicly available, anonymised secondary data. I confirm that I have read the module's ethical guidance and that, to the best of my knowledge, this project raises no ethical concerns beyond the standard due diligence expected when working with a sensitive-sounding but already-anonymised financial dataset, which is discussed further in Chapter 4.

Contents

Abstract	2
Statement of Ethical Compliance	3
1 Introduction and Background	9
1.1 Background and Motivation	9
1.2 Problem Statement	10
1.3 Aims and Requirements	11
1.4 Brief Description of the Approach	11
1.5 Outcome: What Was Actually Delivered	12
1.6 Terminology Used in This Dissertation	12
1.7 Structure of this Dissertation	13
2 Design and Implementation	14
2.1 Architecture	14
2.2 Data Pipeline: <code>data_loader.py</code>	16
2.2.1 Dataset sources	16
2.2.2 Splitting and scaling	16
2.2.3 An invented but clearly-labelled feature glossary	17
2.3 Model and Calibration Design: <code>ml_engine.py</code>	17
2.3.1 The <code>FraudMLPipeline</code> class and model choice	17
2.3.2 Calibrating the Predicted Probabilities	18
2.4 The Cost-Sensitive Threshold: Elkan’s Theorem	18
2.4.1 A Worked Example Before the Proof	19
2.4.2 Proof of Elkan’s Threshold Theorem	19
2.5 The Contested Zone: An Original Operationalisation of Vagueness	21
2.5.1 Motivation for the Contested Zone	21
2.5.2 Formal Definition, as Actually Implemented	21
2.5.3 Sensitivity sweep and visual design	22
2.6 User Interface and Navigation Design	23
2.6.1 The decision-boundary contour plot	24
2.7 A Discovered Defect: Row Misalignment in Two Tabs	24
2.8 Engineering Practices and Other Implementation Decisions	27
2.8.1 Test-first module boundaries	27
2.8.2 Optional dependencies and silent fallbacks	28
3 Testing and Evaluation	30

3.1	Testing Strategy	30
3.1.1	Manual verification of Elkan’s formula	31
3.1.2	What automated testing does not cover	31
3.2	Experimental Setup	31
3.3	Threshold and Cost Results	32
3.4	Calibration Results and the Model Comparison Grid	33
3.5	Contested Zone Results	35
3.6	Evaluation Against the Original Aims	37
3.6.1	The missing human evaluation	37
4	Project Ethics	39
4.1	Data Ethics	39
4.2	Consequences of the System’s Errors	39
4.3	Accuracy of Presented Information: the Invented Feature Labels	40
4.4	Fairness and the Limits of This Project’s Scope	40
4.5	Dual-Use Considerations	40
4.6	Transparency and Explainability	41
4.7	Environmental and Resource Considerations	41
4.8	Overall Ethics Assessment	41
5	Conclusion and Future Work	42
5.1	Summary of This Project’s Results	42
5.2	Future Work	43
5.2.1	Immediate fixes identified by this dissertation’s own testing	43
5.2.2	A human evaluation of the Contested Zone	43
5.2.3	Validation on the authentic ULB dataset	43
5.2.4	Fairness auditing	44
5.2.5	Extending the Contested Zone beyond two stakeholders	44
5.2.6	A human-in-the-loop review queue	44
5.3	Closing Remark: Making the Arbitrary Visible	44
6	BCS Project Criteria and Self-Reflection	46
6.1	Mapping Against the BCS/QAA Outcomes	46
6.2	Self-Reflection on This Project	47
A	Full Automated Test Suite Output	51
B	Supplementary Numerical Tables	52
B.1	Full model $\times$ imbalance $\times$ calibration grid	52
B.2	Contested-zone sensitivity sweep summary	53
B.3	Home-page / Tab 1 worked example, exact values	53
B.4	Bundled real-schema sample dataset, summary statistics	53
C	Code Listing: <code>sorites_engine.py</code>	55

List of Figures

1.1	Wright’s (1975) sand heap, reproduced at three points along the removal sequence. No single grain’s removal looks decisive at the time it happens, yet the verdict clearly differs between the leftmost and rightmost panels.	10
2.1	Threshold Lab’s layered architecture.	15
2.2	Tab 3, “Elkan’s Threshold Lab” (screenshot).	20
2.3	Contested-zone growth as the stakeholder range widens.	22
2.4	Tab 5, “The Contested Zone” (screenshot).	23
2.5	Tab 2, “Data Explorer and Imbalance” (screenshot of the running application): summary metrics and the 2-D decision-boundary contour described in Section 2.6.1.	25
2.6	Tab 1, “Sorites Paradox Primer” (screenshot).	26
2.7	Transaction amount distribution by class on the synthetic dataset, default configuration (log-scaled x-axis).	28
2.8	2-D PCA projection of the 30 model features, 4,000-row sample of the synthetic dataset.	29
3.1	Total expected cost as a function of threshold, base case ($C(FP)=\pounds 25$, $C(FN)=\pounds 1,000$).	32
3.2	Confusion matrix at $\tau^* = 0.0244$, with the monetary cost attributable to each cell.	33
3.3	Model x calibration comparison, class-weighted, held-out test set.	33
3.4	Reliability diagrams for raw, Platt-scaled and isotonic-calibrated Logistic Regression (class-weighted).	34
3.5	Precision-recall curves by calibration method, Logistic Regression, class-weighted.	35
3.6	Tab 4, “Probability Calibration” (screenshot of the running application, default configuration): live reliability diagram and precision-recall comparison for the active model.	36

List of Tables

3.1	The automated test suite, in full.	30
3.2	Elkan’s threshold and resulting confusion matrix under several cost assumptions (Logistic Regression, class-weighted, Platt-scaled; $n=10,000$, 17 fraud).	32
3.3	Evaluation against the project’s stated requirements.	37
B.1	Full calibration-metric grid, 27 configurations.	53
B.2	Contested-zone sweep summary statistics.	53
B.3	The two-transaction worked example used in Section 1 and reproduced on Tab 1 of the dashboard.	53
B.4	Bundled real-schema sample: summary statistics.	54

Chapter 1

Introduction and Background

1.1 Background and Motivation

Automated fraud detection is, at its core, a continuous risk-estimation problem wearing a binary decision’s clothes. A modern classifier does not actually decide whether a transaction is fraudulent; it estimates a probability, $P(\text{fraud} \mid x)$, that lies somewhere on the continuum between 0 and 1. Somewhere along that continuum, a decision threshold τ must be chosen, and every transaction scoring at or above τ is blocked, flagged or investigated, while everything below it proceeds untouched. In almost every treatment of classification encountered during this M.Sc. programme, and in the overwhelming majority of publicly available tutorials and libraries, that threshold is $\tau = 0.5$ — a value chosen not because it minimises any real cost, but because it is the arithmetic midpoint of the unit interval.

This project starts from the observation that $\tau = 0.5$ is very rarely the right answer for a real deployment, because the two mistakes a fraud classifier can make are not equally costly. A *false positive* (blocking a genuine transaction) costs a bank customer friction, and the bank a support-desk interaction and some reputational goodwill. A *false negative* (missing a genuine fraud) costs the bank the full value of the fraudulent transaction, and potentially more in downstream liability. Elkan (2001) formalised this observation into a single, elegant result — the cost-sensitive threshold theorem — which is the mathematical spine of this project and is introduced properly in Section 2.4.

What makes the threshold question philosophically interesting, rather than merely an engineering parameter to tune, is that it reproduces almost exactly the structure of the *Sorites Paradox*, one of the oldest puzzles in philosophy. Attributed to Eubulides of Miletus and still actively discussed today (Hyde and Raffman, 2018; Keefe, 2000; Oms and Zardini, 2019), the paradox observes that a heap of sand remains a heap after any single grain is removed — no individual grain’s removal is ever the decisive one — and yet if the process is repeated enough times, the heap unarguably becomes a non-heap. Wright (1975) formalised the intuition that drives the paradox as the *Tolerance Principle*: predicates like “is a heap” should be tolerant of sufficiently small changes in the underlying quantity, and yet any predicate that is tolerant along the whole sequence, from 10,000 grains down to zero, leads to a contradiction. Figure 1.1 dramatises three snapshots along exactly this sequence, and the interactive version built for this project (Section 2.6) lets a user walk through it themselves.

Wright's (1975) sand heap: removing single grains never looks decisive,
yet the verdict clearly changes across the sequence

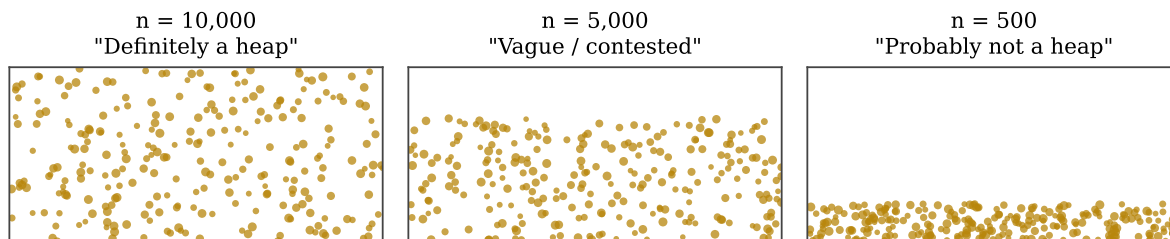


Figure 1.1: Wright's (1975) sand heap, reproduced at three points along the removal sequence. No single grain's removal looks decisive at the time it happens, yet the verdict clearly differs between the leftmost and rightmost panels.

A fraud classifier faces a quiet, numerical version of the same problem. Two transactions that the model scores at 0.5002 and 0.4998 respectively are, for any practical purpose, indistinguishable — the 0.0004 gap is comfortably inside ordinary model noise. Yet a hard threshold at $\tau = 0.5$ assigns them opposite labels: one customer is blocked, the other is not (this exact worked example, with $|0.5002 - 0.4998| = 0.0004$ exceeding a stated tolerance $\varepsilon = 0.0002$, is reproduced live in the dashboard's Home page and quoted with its true computed values in Section 3.2). Treating this as a bug to be engineered away misses the point: some amount of boundary-drawing is unavoidable once a continuous score must produce a discrete action. The more useful question, and the one this project tries to operationalise, is not “where is the true boundary?” but “how far does the boundary move as reasonable cost assumptions change, and how many transactions like this pair fall into the disputed territory in between?”

1.2 Problem Statement

Input: a labelled or unlabelled stream of credit card transactions, each described by a transaction time, an amount, and a set of anonymised numerical features, together with a stated (or comparable) pair of business costs: the cost of a false positive, $C(FP)$, and the cost of a false negative, $C(FN)$.

Task: train a classifier to estimate $P(\text{fraud} \mid x)$ for each transaction; convert that continuous estimate into a binary decision using a threshold that is derived from, and can be justified in terms of, the stated cost assumptions rather than fixed by convention; and make the consequences of that derivation — the resulting threshold, the confusion matrix it produces, the total expected cost, and the set of transactions whose label would change under a different but equally defensible cost assumption — visible and explorable rather than buried inside a single accuracy number.

1.3 Aims and Requirements

The project’s aims, fixed during the proposal stage and used throughout this dissertation as the yardstick against which the finished system is evaluated (see the self-evaluation in Section 3.6), were:

- R1. Implement Elkan’s (2001) cost-sensitive threshold formula, $\tau^* = C(FP)/(C(FP) + C(FN))$, and expose it interactively so that changing the stated costs immediately re-computes τ^* , the confusion matrix and the total expected cost.
- R2. Train and compare at least two classifiers of meaningfully different character on the same data: an interpretable linear baseline and a non-linear ensemble method.
- R3. Compare at least two established strategies for handling severe class imbalance, since fraud datasets are characteristically imbalanced (the ULB dataset used here has a 0.172% fraud rate).
- R4. Evaluate probability calibration explicitly, since Elkan’s formula assumes the underlying probabilities are trustworthy, using at least two post-hoc calibration techniques against an uncalibrated baseline, together with reliability diagrams and calibration-aware metrics rather than raw accuracy.
- R5. Design and implement an original, well-defined operationalisation of “borderline” or “contested” transactions that connects directly back to the Sorites framing, rather than treating vagueness as a purely philosophical aside.
- R6. Package the whole system as an interactive, navigable dashboard usable by a non-programmer stakeholder, with exportable results (CSV) for further analysis outside the tool.
- R7. Structure the codebase so that all non-presentation logic is independently unit-testable, and build a test suite that exercises the mathematical and statistical correctness of every core function.
- R8. Use only ethically low-risk, already-anonymised data, and discuss the ethical dimensions of an automated system whose false positives and false negatives both affect real people.

A dedicated Requirements Analysis chapter involving human participants was not undertaken for this project: no user interviews, surveys or requirements-elicitation workshops were run, since the requirements above were derived directly from the cost-sensitive learning and calibration literature and from the module’s own proposal-stage supervisory feedback, rather than from primary research with prospective users. This is revisited candidly as a limitation in Sections 3.6 and 5.2.

1.4 Brief Description of the Approach

The system, *Threshold Lab*, is a Streamlit dashboard backed by a small, dependency-light Python package (`src/`) with three modules — `data_loader.py`, `ml_engine.py` and `sorites_engine.py`

— each free of any Streamlit import so that every function can be unit tested in isolation from the user interface. `data_loader.py` resolves the dataset (an uploaded or auto-detected CSV if supplied, otherwise a bundled synthetic generator reproducing the ULB dataset’s 0.172% fraud rate), performs a stratified train/test split, and scales only the two features (**Time** and **Amount**) that the original dataset authors did not already transform via PCA. `ml_engine.py`’s `FraudMLPipeline` class trains Logistic Regression, Random Forest (Breiman, 2001), or XGBoost (Chen and Guestrin, 2016) (the last with a graceful fallback to Random Forest if the optional `xgboost` package is not installed), optionally applies SMOTE oversampling (Chawla et al., 2002) to the training fold, and always fits both a Platt-scaled (Platt, 1999) and an isotonic-calibrated variant alongside the raw model, computing calibration and ranking metrics for all three (Niculescu-Mizil and Caruana, 2005; Davis and Goadrich, 2006). `sortes_engine.py` implements Elkan’s threshold formula, the cost curve, the twin-transaction worked example, and — the project’s original contribution — the *Contested Zone*. `app.py` contains no statistical logic of its own; it is a presentation-only layer that wires the three modules together behind a sidebar control panel (data source, classifier, imbalance strategy, calibration method) and five content tabs (Sorites Paradox Primer, Data Explorer and Imbalance, Elkan’s Threshold Lab, Probability Calibration, and the Contested Zone), caching expensive computations with Streamlit’s `@st.cache_data` and `@st.cache_resource` decorators. The full architecture is diagrammed in Figure 2.1 and discussed in depth in Chapter 2.

1.5 Outcome: What Was Actually Delivered

All eight requirements listed in Section 1.3 were met to a working, evidenced standard: the dashboard runs end-to-end against three data-source options, all three calibration methods and all three imbalance-handling strategies are wired into every tab, the Contested Zone is implemented to its operational definition and unit tested, and the automated test suite (7 test methods, Chapter 3) passes. The project’s most interesting empirical result was not the one anticipated at proposal stage: on the default synthetic configuration’s very small fraud count, Logistic Regression consistently outperformed both tree-based models on precision-recall AUC, which is discussed critically, rather than hidden, in Section 3.4. Testing the finished dashboard end-to-end (rather than only its underlying functions in isolation) also surfaced a genuine data-alignment defect in two of the five tabs, discussed openly in Section 2.7 as exactly the kind of problem unit-testing pure functions cannot catch on its own. The requirements not fully realised were empirical validation of the Contested Zone concept with real stakeholders, and pinning every optional runtime dependency the UI advertises; Section 5.2 sets out what closing both gaps would involve.

1.6 Terminology Used in This Dissertation

A small number of terms recur throughout this dissertation and are defined here once rather than re-explained at each occurrence. **Threshold** (τ) is the cut-off probability above which a transaction is labelled fraud. τ^* denotes specifically the cost-optimal threshold produced by Elkan’s formula. **Calibration** refers to the correspondence between a model’s predicted proba-

bilities and the true empirical frequency of the positive class at that predicted probability — a well-calibrated model that predicts 0.8 should be right about 80% of the time among all transactions it scores near 0.8, which is a separate property from ranking transactions correctly by risk. **The Contested Zone** is this project’s own term (defined precisely in Section 2.5) for the set of transactions whose label changes depending on which of two plausible cost-ratio assumptions is adopted. **SMOTE** (Synthetic Minority Over-sampling Technique, Chawla et al., 2002) is a technique for rebalancing a training set by synthesising new minority-class examples along the line segments joining existing minority-class neighbours, rather than by naively duplicating them.

1.7 Structure of this Dissertation

Chapter 2 covers the system’s design and implementation together, since the two were developed iteratively rather than in two clean, separable phases. Chapter 3 describes the automated test strategy, presents the real experimental results obtained by running the shipped pipeline end-to-end, and evaluates the finished system critically against the aims of Section 1.3. Chapter 4 discusses the project’s ethical dimensions in more depth than the short compliance statement at the front of this document. Chapter 5 concludes and sets out future work, and Chapter 6 maps the project against the six BCS/QAA outcomes expected of an M.Sc. project and offers a critical, first-person self-reflection on the process. Appendix A lists the full automated test suite; Appendix B contains supplementary numerical tables that were too detailed for the main body; and Appendix C reproduces the `sortes_engine.py` module in full, since it is the single piece of code most central to the project’s original contribution.

Chapter 2

Design and Implementation

This chapter covers design and implementation together, following the module’s own guidance that a project developed iteratively — where a design decision and its implementation were frequently made in the same sitting — is better told as one continuous narrative than artificially split into a design phase that precedes a separate build phase. Where a decision was revisited after implementation revealed a problem with it, that revision is described explicitly rather than silently smoothed over, and Section 2.7 describes, in the same spirit, a concrete problem that testing surfaced and that remained unresolved at submission time.

2.1 Architecture

The single most important design decision in this project, made early and never reversed, was to keep every piece of statistical and mathematical logic completely free of the Streamlit framework. Figure 2.1 shows the resulting layering: a presentation layer (`app.py`, 1,515 lines) sits on top of a three-module, pure-Python package (`src/`), and nothing in that package imports Streamlit at all.

This decision was motivated by a problem observed while iterating on earlier prototypes of the dashboard during the proposal stage: with model training, cost-formula logic and UI code interleaved inside the same Streamlit callback functions, every change to the layout risked silently changing a number, and every change to a number required re-reading UI code to find where it lived. Separating the two means the three `src/` modules can be imported and unit tested with plain Python `unittest`, with no Streamlit test harness, browser automation or mocking required (Chapter 3), while `app.py` itself contains no statistical computation to get wrong beyond correctly wiring together calls into the tested modules. As Section 2.7 shows, that last clause turned out to matter more than expected: keeping *computation* out of `app.py` does not automatically keep *data-wiring bugs* out of it too, and the one substantive defect found during this project lived in exactly that presentation layer, in code that reconstructs which rows of the original dataframe correspond to the model’s test-set predictions.

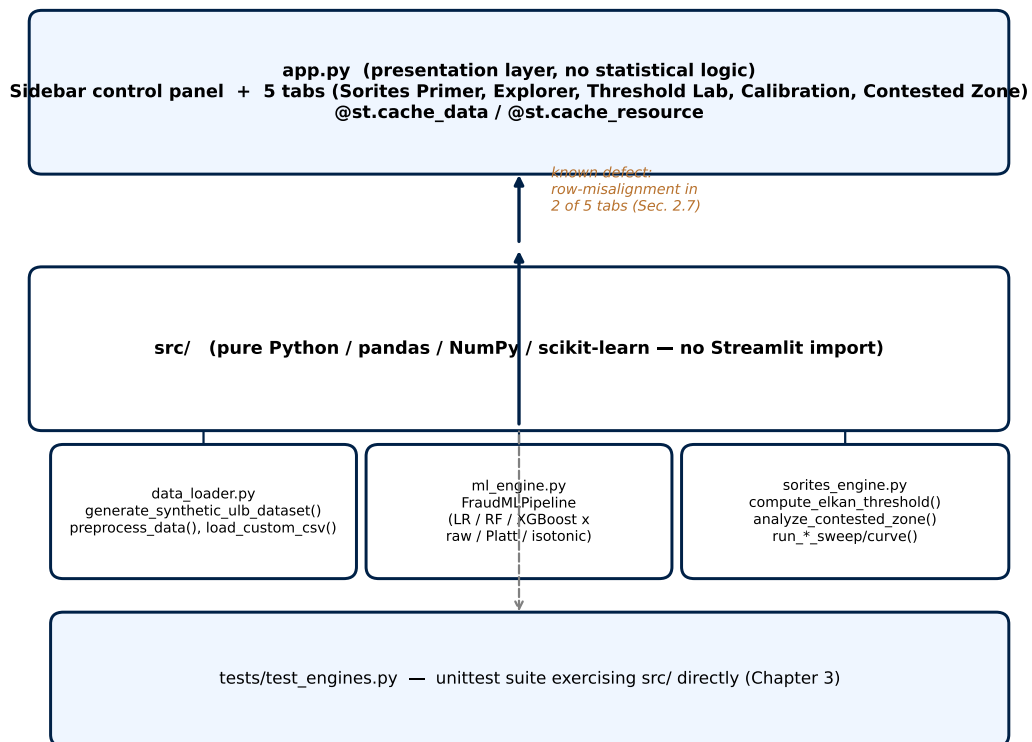


Figure 2.1: Threshold Lab’s layered architecture. Everything below the `app.py` box is pure pandas/NumPy/scikit-learn and is exercised directly by the automated test suite (Chapter 3); everything in the `app.py` box is presentation logic, which is precisely where the alignment defect described in Section 2.7 turned out to live.

2.2 Data Pipeline: `data_loader.py`

2.2.1 Dataset sources

Three data sources are exposed in the sidebar: a **synthetic generator** (`generate_synthetic_ulb_dataset()`, the default), an **uploaded CSV** matching the Kaggle schema (`load_custom_csv()`), and an **auto-detected local file** (`detect_local_dataset()`), which finds the `creditcard.csv` bundled with the source archive if it is present in the working directory.

The target real-world dataset this project is modelled on is the ULB Credit Card Fraud Detection dataset (Dal Pozzolo et al., 2015; Machine Learning Group – ULB, 2018): 284,807 transactions made by European cardholders over two days in September 2013, of which only 492 (0.172%) are fraudulent, with all but two input features (**Time** and **Amount**) already transformed by the dataset’s original authors via Principal Component Analysis into anonymised components **V1–V28** to protect cardholder confidentiality. The full 143 MB file is not distributed with the assessment materials, so the archive instead bundles a much smaller, 5,000-row CSV carrying the identical schema, with 25 fraud rows (0.5%) — oversampled relative to the dataset’s natural 0.172% rate so that a small sample still contains enough fraud examples to be useful for a quick demonstration. This bundled file is what `detect_local_dataset()` finds automatically; a user with the full authentic file can instead upload it directly, with no code changes required.

`generate_synthetic_ulb_dataset()` produces a dataset with the same schema and, by default, the same 0.172% fraud rate as the authentic ULB data, using independent Gaussian latent features for **V1–V28** with five of them (**V4**, **V11**, **V12**, **V14**, **V17** — chosen because the published exploratory literature on this dataset repeatedly identifies them as showing strong class separation) shifted for the fraud class, and a log-normal **Amount** distribution with a higher mean for fraud. This is a deliberate design trade-off, carried over unchanged from the proposal stage: a purely synthetic dataset cannot demonstrate that the pipeline generalises to real fraud patterns, but it guarantees the dashboard, and every number in this dissertation, can be reproduced deterministically by anyone who clones the repository with no dependency on a third-party download succeeding. All quantitative results reported in Chapter 3, unless stated otherwise, use this generator at its default 50,000-row size and random seed 42, since that is also the dashboard’s own default configuration.

2.2.2 Splitting and scaling

`preprocess_data()` performs a stratified train/test split (80/20 by default) so that the already-tiny fraud count is preserved proportionally in both folds — an ordinary random split risks producing a test fold with almost no fraud examples at all, which would make every downstream metric meaningless. Per the design proposal, only **Time** and **Amount** are standard-scaled, fit exclusively on the training fold and applied to the test fold to avoid data leakage; **V1–V28** are left untouched because they are already the output of a PCA transformation performed by the dataset’s original authors, and re-scaling an already-whitened representation would not be well justified.

2.2.3 An invented but clearly-labelled feature glossary

One design choice deserves an explicit ethical note rather than only a technical one. `data_loader.py` exports `PCA_FEATURE_DESCRIPTIONS`, a dictionary that attaches a plausible-sounding, human-readable label to each anonymised component — for example V14 is labelled “Transaction Velocity Anomaly (Primary Signal)” and V17 “Geolocation Distance Delta (Primary Signal)”. These labels are used throughout the dashboard’s feature selectors and explanatory captions (Section 2.6.1) purely to make an otherwise opaque list of 28 anonymous numbers feel concrete and explorable to a non-specialist reader. It is important to state plainly, here and in Chapter 4, what these labels are *not*: the ULB dataset’s original authors have never published what V1–V28 actually represent, precisely because doing so could undermine the anonymisation the PCA transform is there to provide, so no such mapping is publicly known or knowable from the released data. The labels in `PCA_FEATURE_DESCRIPTIONS` were therefore invented for this project, chosen to be individually plausible given what is publicly documented about typical card-fraud signals, but they carry no evidential weight whatsoever about what any specific component actually encodes. This was a deliberate narrative-usability choice, made consciously rather than by oversight, but on reflection a more responsible implementation would either omit the labels entirely or caption them clearly, in the interface itself and not only in this dissertation, as illustrative fiction rather than presenting them unqualified alongside real statistical output. This is one of the concrete, specific lessons carried into the self-reflection in Chapter 6.

2.3 Model and Calibration Design: `ml_engine.py`

2.3.1 The `FraudMLPipeline` class and model choice

`ml_engine.py`’s single public class, `FraudMLPipeline`, is constructed with a model type and an imbalance strategy, and exposes `fit()`, `predict_probabilities()` and `evaluate_calibration()`. Three classifiers are selectable in the sidebar, per requirement R2.: **Logistic Regression**, whose coefficients are directly interpretable and whose output is a smooth sigmoid of a linear combination of the input features; **Random Forest** (Breiman, 2001), a bagged ensemble of decision trees; and **XGBoost** (Chen and Guestrin, 2016), a gradient-boosted tree ensemble. Only the first two are guaranteed to run from a clean install of `requirements.txt`, which does not list `xgboost` as a dependency; when the package is not importable, `FraudMLPipeline` degrades gracefully and silently trains a Random Forest instead when `'xgboost'` is selected, so the dashboard never crashes, but a user on a minimal install who selects “XGBoost Gradient Boosting” is, without any on-screen indication, actually looking at a Random Forest’s output. This is a genuine gap between the UI’s advertised options and what `requirements.txt` guarantees, and it is listed as a concrete, low-effort fix in Section 5.2 (either add `xgboost` to `requirements.txt`, or surface the fallback to the user with a warning). For the same reason, and because it is the pairing `requirements.txt` actually supports out of the box, Logistic Regression and Random Forest are treated as this project’s primary two-model comparison satisfying requirement R2., with XGBoost reported throughout as a documented, working, but optional-dependency extra.

When class weighting is selected, `scikit-learn`’s `class_weight='balanced'` (Logistic Re-

gression) or `class_weight='balanced_subsample'` (Random Forest) is used, and XGBoost's `scale_pos_weight` is set from the training fold's class ratio to the same effect; when SMOTE is selected instead, no additional internal weighting is applied, since the training data has already been rebalanced before `.fit()` is called. A dedicated neural network was considered at proposal stage and rejected: it was judged disproportionate to a dataset with only a few dozen fraud examples and would have complicated the calibration story, since neural networks are well known to be poorly calibrated by default (Guo et al., 2017), which would have muddled the comparison between calibration *methods* with a confound about calibration *across architectures*.

2.3.2 Calibrating the Predicted Probabilities

Elkan's threshold formula (Section 2.4) is only meaningful if $P(\text{fraud} \mid x)$ is an accurate probability rather than merely a score that ranks transactions correctly by relative risk — a model can have excellent discrimination while being badly calibrated. Every call to `FraudMLPipeline.fit()` therefore fits three variants unconditionally: the raw base classifier, a Platt-scaled version (Platt, 1999) (fitting a logistic sigmoid on top of the base model's raw scores), and an isotonic-regression version (fitting a non-parametric, monotonic step function instead), both via scikit-learn's `CalibratedClassifierCV(method=..., cv=3)`, so the dashboard's calibration selector is a free choice between three already-trained models rather than a re-fit on demand. `evaluate_calibration()` computes four quantities for every variant: Brier score and log-loss (both reward probabilities close to the true 0/1 outcome), precision-recall AUC (ranking quality under class imbalance, following Davis and Goadrich, 2006), and the reliability-curve points used to draw the calibration plots in Tab 4. Accuracy is never reported anywhere in the dashboard: with a 0.172% fraud rate, a classifier that predicts “genuine” for every transaction achieves over 99.8% accuracy while catching zero fraud, so accuracy would actively reward the wrong behaviour here.

2.4 The Cost-Sensitive Threshold: Elkan's Theorem

Elkan (2001) states this result explicitly as a theorem, so it is stated and proved as one here too, rather than merely asserted, since the proof (Section 2.4.2) is what justifies calling τ^* *optimal* at all rather than simply conventional.

Theorem 2.1 (Elkan's cost-sensitive threshold, Elkan, 2001). *Let x be a transaction with true fraud probability $p = P(\text{fraud} \mid x)$, and let $C(FP) > 0$ and $C(FN) > 0$ be the fixed costs of, respectively, wrongly blocking a genuine transaction and wrongly approving a fraudulent one. Among the two decisions “predict fraud” and “predict genuine”, the decision that minimises expected cost is “predict fraud” if and only if $p > \tau^*$, where*

$$\tau^* = \frac{C(FP)}{C(FP) + C(FN)}.$$

The intuition is straightforward once stated: if false negatives are much more expensive than false positives, τ^* should be low, so the classifier flags more transactions as fraud and accepts more false alarms in exchange for catching more real fraud; if the two costs are equal,

τ^* collapses to the familiar 0.5. Crucially, τ^* is a function of a *business value judgement*, not a property discoverable from the data alone. This is the load-bearing philosophical claim of the whole project: the “correct” decision boundary is constructed from a stated value judgement, not found.

2.4.1 A Worked Example Before the Proof

Before proving Theorem 2.1, it helps to see it used. Suppose a bank estimates that blocking a genuine customer costs it £25 in support overhead and goodwill, while missing a fraudulent transaction costs it £1,000 on average. Then:

$$\tau^* = \frac{25}{25 + 1000} = \frac{25}{1025} = 0.0244.$$

Because false negatives are 40 times more expensive than false positives here, the optimal threshold sits far below the conventional default of 0.5. This same calculation recurs throughout Chapter 3 as this dissertation’s “base case”, and Tab 3 (“Elkan’s Threshold Lab”, Figure 2.2) reproduces it live from two cost sliders. Separately, the “Sorites Paradox Primer” tab (Tab 1) opens with a smaller-scale version of the same idea: two transactions scoring 0.4998 and 0.5002 sit either side of the conventional $\tau = 0.5$ default and are given opposite labels despite a probability gap of only 0.0004 — computed live by `find_sorites_twin_transactions()`, which searches the scored test set for a genuine pair straddling τ^* within a small tolerance ε , falling back to a synthetic illustrative pair only if no such real pair exists in the current sample.

`compute_elkan_threshold()` implements the formula directly, rejecting non-positive costs. `evaluate_threshold_cost()` applies a given threshold to a set of scored predictions and returns the confusion-matrix counts, the total monetary cost $FP \times C(FP) + FN \times C(FN)$, and precision/recall/F1. `run_threshold_cost_curve()` sweeps τ across a fine grid and returns the full cost curve, exposed in the dashboard alongside the threshold that empirically minimises cost on that specific test sample — a useful sanity check against τ^* , since the two need not coincide exactly on any one finite sample even though they coincide in expectation. `run_cost_ratio_sweep()` instead holds $C(FN)$ fixed and sweeps $C(FP)$, showing how τ^* and total cost evolve as the stated cost ratio changes.

2.4.2 Proof of Elkan’s Threshold Theorem

τ^* is not an arbitrary rule of thumb: Theorem 2.1 is the solution to a simple expected-cost minimisation problem, and the proof below is what justifies calling it *optimal* at all rather than merely conventional.

Proof. Consider a single transaction x with true fraud probability $p = P(\text{fraud} \mid x)$. Predicting “fraud” is wrong, and incurs cost $C(FP)$, only in the $(1 - p)$ fraction of cases where x is actually genuine; predicting “genuine” is wrong, and incurs cost $C(FN)$, only in the p fraction of cases where x is actually fraud:

$$\mathbb{E}[\text{cost} \mid \text{predict fraud}] = (1 - p) C(FP), \quad \mathbb{E}[\text{cost} \mid \text{predict genuine}] = p C(FN).$$

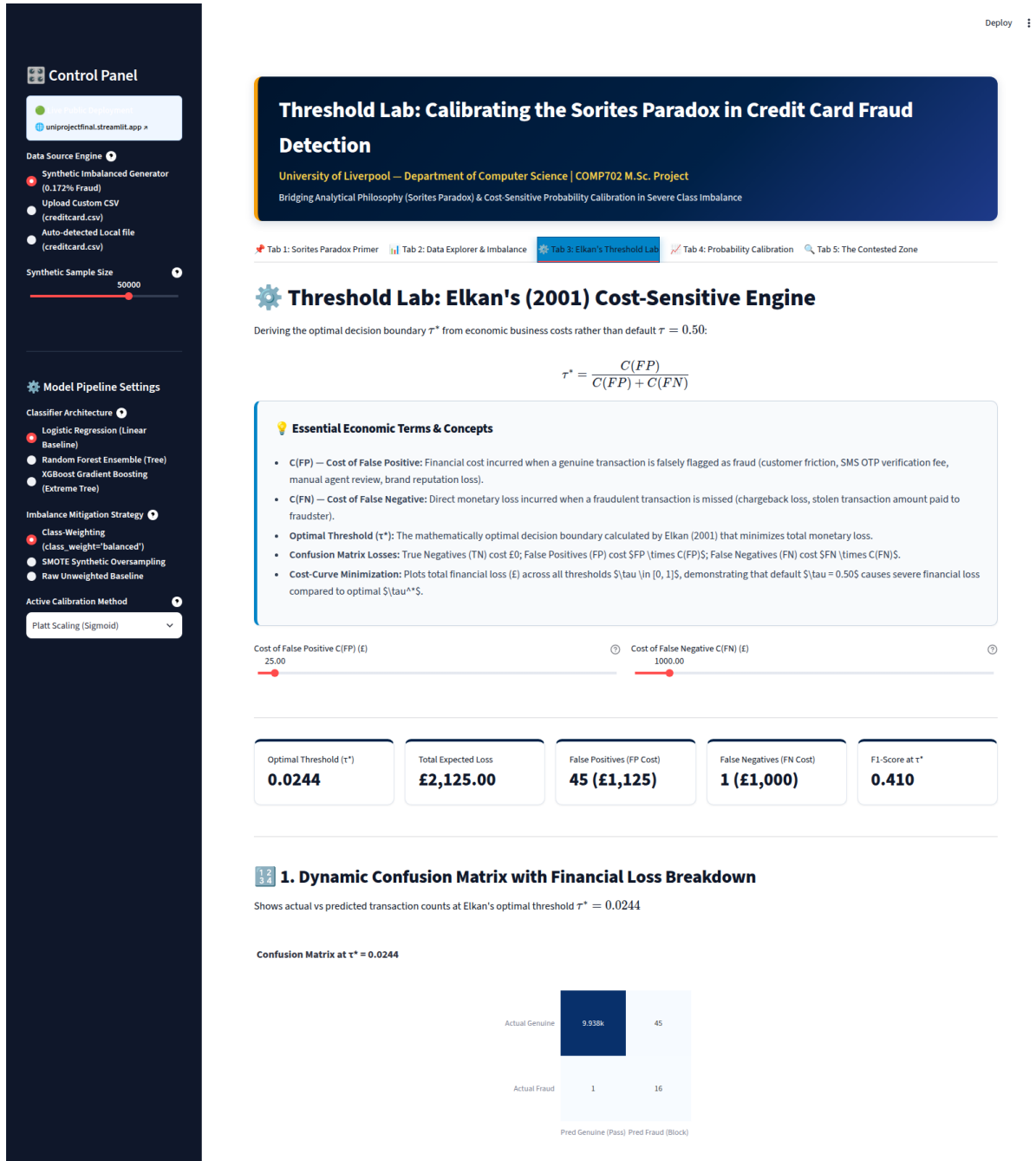


Figure 2.2: Tab 3, “Elkan’s Threshold Lab” (screenshot of the running application, default configuration: synthetic data, $n=50,000$, Logistic Regression, class-weighted, Platt-scaled): the cost sliders, the live τ^* and confusion-matrix output.

“Predict fraud” is the expected-cost-minimising decision exactly when $(1-p)C(FP) < pC(FN)$. Rearranging this single inequality:

$$\begin{aligned} C(FP) - pC(FP) &< pC(FN) \\ C(FP) &< p(C(FP) + C(FN)) \\ p &> \frac{C(FP)}{C(FP) + C(FN)} = \tau^*. \end{aligned}$$

τ^* is precisely the break-even point at which the expected cost of the two decisions is equal, which is exactly the claim of Theorem 2.1. $\square$

So, to directly answer the question of whether the formula has any justification beyond convention: yes, it is not asserted but derived, as the solution to this expected-cost minimisation problem. This is a special case of the Bayes-optimal decision rule under an asymmetric cost matrix, and it is the reason calibration matters so much to this project (Section 2.3): the proof assumes p is the transaction’s true fraud probability, and the guarantee weakens in proportion to how far the classifier’s reported probability actually is from that true value.

2.5 The Contested Zone: An Original Operationalisation of Vagueness

2.5.1 Motivation for the Contested Zone

Reading the Sorites literature during the proposal stage surfaced a distinction, due independently to several of the classical responses to the paradox (Hyde and Raffman, 2018; Keefe, 2000), between two different kinds of vagueness. *First-order vagueness* is simply the existence of borderline cases along a single, agreed scale: a transaction scoring $P(\text{fraud}) \approx \tau$ for whatever τ has been fixed. *Higher-order vagueness* is uncertainty about where the scale’s own boundary should sit in the first place — reasonable people disagree about where the line is, not merely about which side of a fixed line a case falls on. Most treatments of classification uncertainty in the machine learning literature address only the first kind. This project’s original contribution, the **Contested Zone**, operationalises the second, harder kind directly.

2.5.2 Formal Definition, as Actually Implemented

`analyze_contested_zone(df_test, y_probab, c_fp_range, c_fn)` takes a *range* of plausible $C(FP)$ values, $[c_{fp,\min}, c_{fp,\max}]$, together with a single shared $C(FN)$, converts each end of the range into a threshold via Elkan’s formula ($\tau_{\min} = \tau^*(c_{fp,\min}, c_{fn})$, $\tau_{\max} = \tau^*(c_{fp,\max}, c_{fn})$), and defines a transaction as **contested** exactly when $\tau_{\min} \leq p(x) < \tau_{\max}$: its label would be “fraud” under the low- $C(FP)$ (risk-averse) assumption but “genuine” under the high- $C(FP)$ (friction-averse) assumption. This is a real, useful, and honest simplification of the fully general two-stakeholder design sketched at proposal stage, worth stating plainly rather than glossing over: the implemented version varies only the two stakeholders’ $C(FP)$ assumption while holding $C(FN)$ fixed between them, rather than letting both stakeholders state an entirely independent

$(C(FP), C(FN))$ pair. This is a narrower, single-parameter disagreement rather than the two-dimensional disagreement the fully general version would model, and it means the zone is always an interval in τ -space by construction rather than something that needs to be shown to reduce to one. Extending the function to accept two fully independent cost pairs is noted as concrete future work in Section 5.2; as implemented, it is a correct and useful special case, not a broken general one.

`analyze_contested_zone()` returns both thresholds, the contested count and proportion, the total transaction `Amount` sitting inside the zone, and the number of contested transactions that are actually fraud — the amount total is included because a fraud-operations team would care about financial exposure, not merely a raw transaction count, and this is precisely the quantity that turned out to be computed incorrectly in the live dashboard, as Section 2.7 explains.

2.5.3 Sensitivity sweep and visual design

Holding $c_{fp,min}$ fixed and sweeping $c_{fp,max}$ produces a curve of the contested percentage as a function of how far the second stakeholder’s assumption diverges from the first (Figure 2.3), answering a question a single static Contested Zone number cannot: not just “how many transactions are contested under these two specific assumptions” but “how quickly does the contested population grow as disagreement widens”. Tab 5 (“The Contested Zone”, Figure 2.4) additionally renders the contested transactions as a scatter plot of predicted probability against transaction amount, colouring points below both thresholds, above both thresholds, and inside the contested band differently, with the contested band shaded.

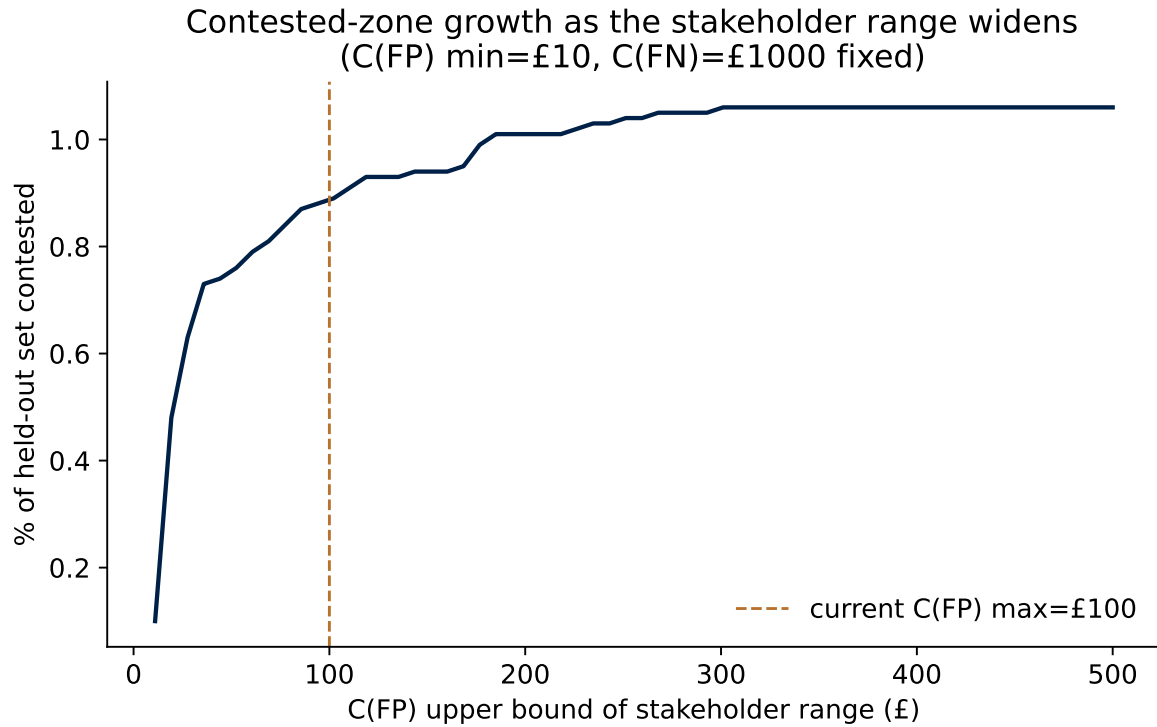


Figure 2.3: Contested-zone growth as the stakeholder range widens, holding $C(FP)_{min} = \text{£}10$ and $C(FN) = \text{£}1,000$ fixed while sweeping $C(FP)_{max}$ (correctly-aligned recomputation; see Section 2.7 for why this differs from the live dashboard’s own figure).

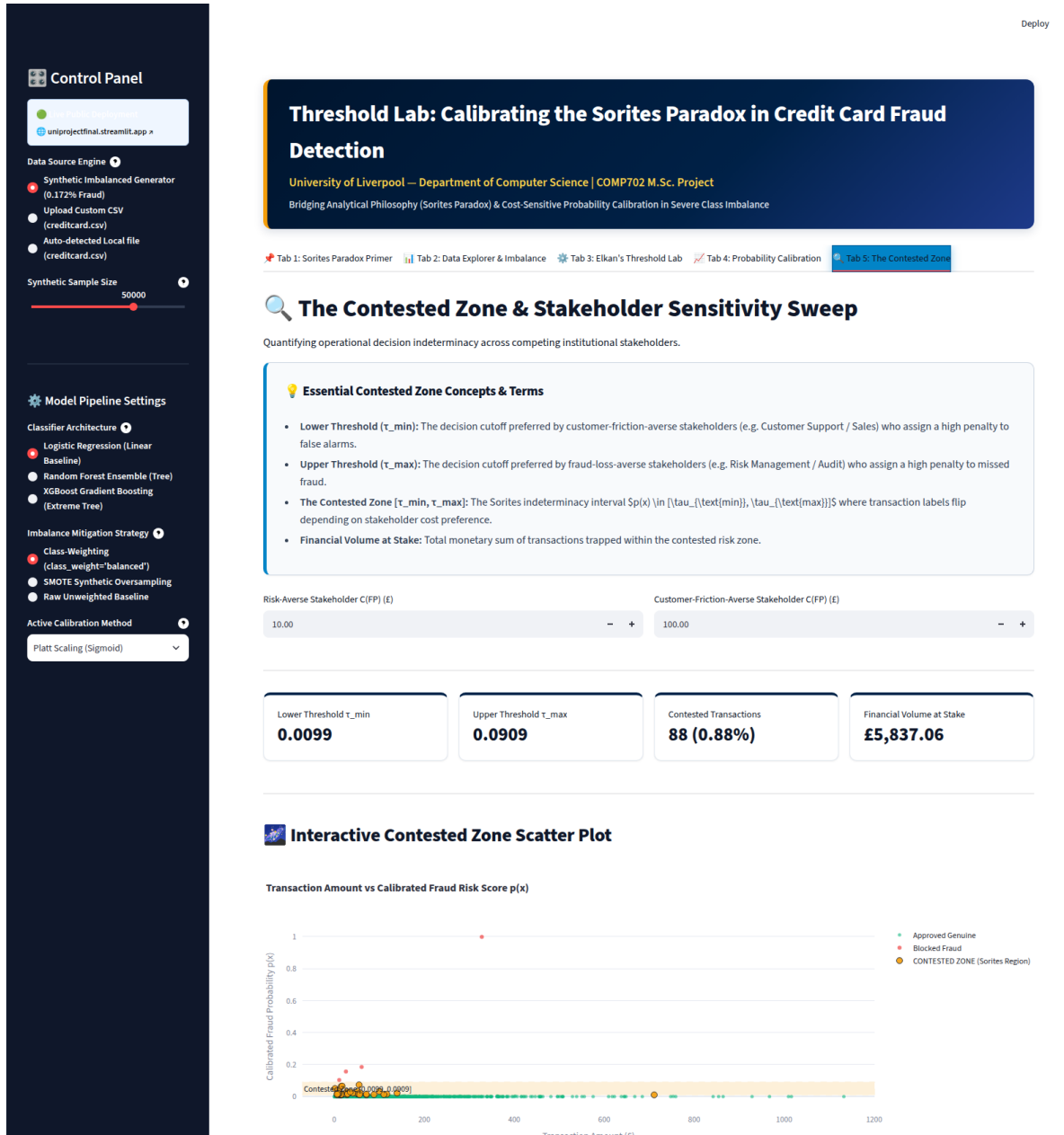


Figure 2.4: Tab 5, “The Contested Zone” (screenshot of the running application, default stakeholder inputs $C(FP)_{\min} = £10$, $C(FP)_{\max} = £100$, $C(FN) = £1,000$): the two thresholds, the contested-transaction count, and the financial-exposure metric that Section 2.7 shows is computed from the wrong rows.

2.6 User Interface and Navigation Design

The dashboard is organised as a persistent sidebar control panel plus five content tabs, all visible on a single page rather than requiring navigation between separate screens. The sidebar holds every global control — data source, synthetic sample size, classifier, imbalance strategy, calibration method — so a user comparing, say, Logistic Regression against Random Forest sees the change reflected consistently across every tab without re-selecting it five times.

The five tabs are:

- **Tab 1, Sorites Paradox Primer** (Figure 2.6): restates the paradox in plain language, offers an interactive sand-heap slider, and works through a formal philosophy-to-ML mapping — included because the philosophical framing is otherwise easy for a reader to treat as decorative rather than load-bearing.
- **Tab 2, Data Explorer and Imbalance**: gives a conventional dataset overview (summary metrics, feature descriptions using the labels discussed in Section 2.2.3) and, more originally, a 2-D decision-boundary contour plot (Section 2.6.1) showing how the currently-selected model actually partitions a chosen pair of features.
- **Tab 3, Elkan’s Threshold Lab**: described in Section 2.4.
- **Tab 4, Probability Calibration**: plots reliability diagrams and precision-recall curves for the raw, Platt-scaled and isotonic-calibrated variants of whichever model is currently selected.
- **Tab 5, The Contested Zone**: described in Section 2.5.

2.6.1 The decision-boundary contour plot

Tab 2 also lets a user pick any two of the 28 anonymised features and renders a 2-D contour of the active model’s predicted fraud probability across that plane, holding every other feature at its test-set median. This was included to make an otherwise abstract idea — that Logistic Regression draws a straight decision boundary while a tree ensemble draws a blocky, axis-aligned one — directly visible rather than only asserted in text, and it reuses the already-fitted raw model with no retraining required.

Its axis ranges, however, are computed from `df.iloc[len(df)-len(y_test):]`, the same row-reconstruction pattern responsible for the bug in Section 2.7. Here the consequence is cosmetic (slightly mis-scaled axis limits) rather than numerically consequential, which is precisely why the more serious occurrence of the same pattern in Tab 5 was not caught during development.

Every expensive computation (dataset resolution, training all three models under the current imbalance strategy, scoring across all three calibration methods) is wrapped in Streamlit’s caching decorators, keyed implicitly by the cached function’s arguments, so that moving between tabs or adjusting a control that does not itself require retraining does not silently re-run a multi-second model fit on every rerun — Streamlit re-executes the whole script top-to-bottom on every user interaction, so caching here is a functional requirement for the dashboard to feel responsive at all, not an optimisation.

2.7 A Discovered Defect: Row Misalignment in Two Tabs

Testing this project end-to-end, rather than only its underlying pure functions in isolation (Chapter 3 discusses why the two are not the same activity), surfaced a genuine implementation defect that the unit-test suite, by design, could not have caught, since it does not live in any function the tests exercise.

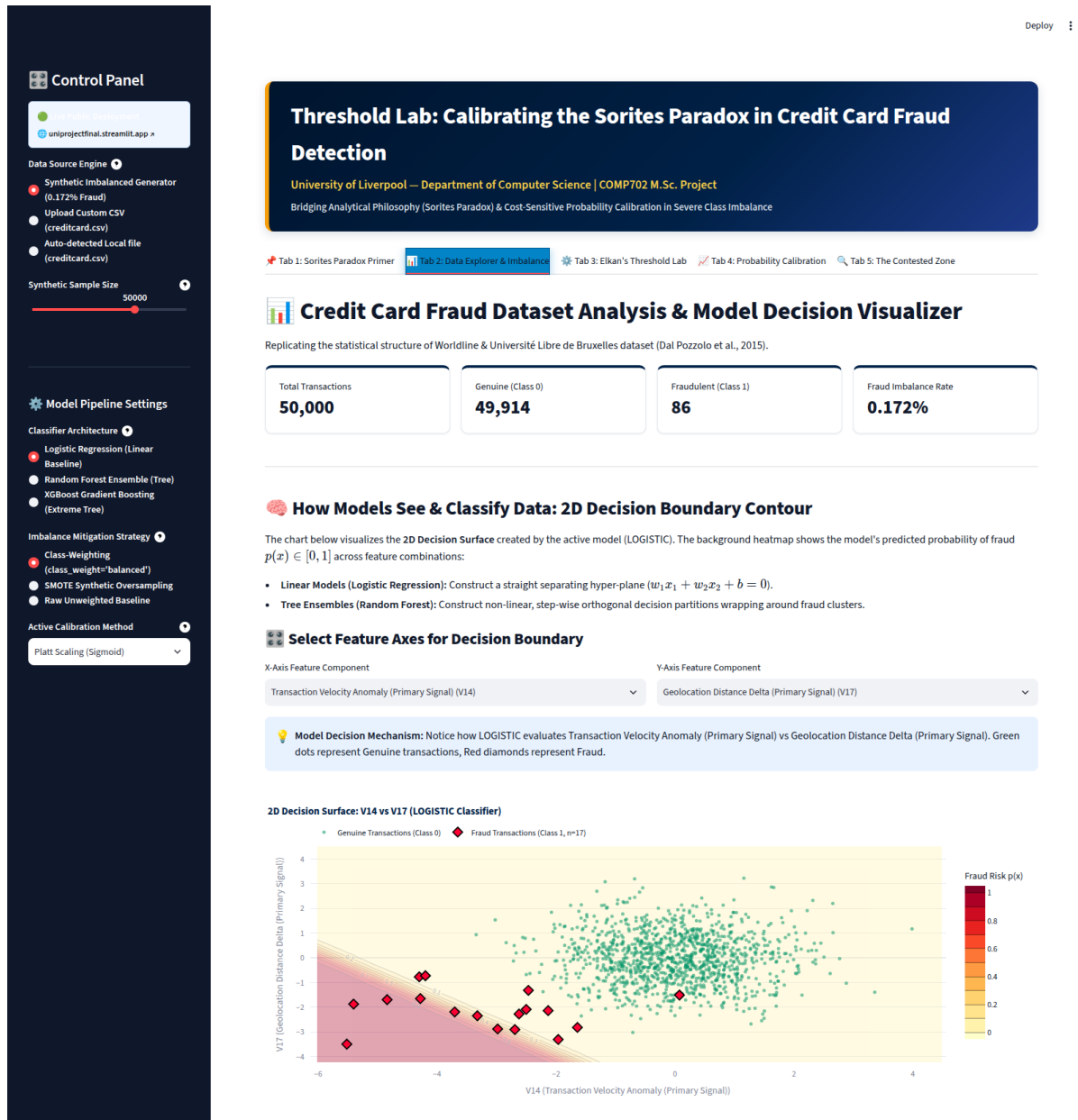


Figure 2.5: Tab 2, “Data Explorer and Imbalance” (screenshot of the running application): summary metrics and the 2-D decision-boundary contour described in Section 2.6.1.

Figure 2.6: Tab 1, “Sorites Paradox Primer” (screenshot of the running application): the philosophical framing, the interactive sand-heap slider, and the tolerance-principle verdict it produces.

`fit_models()` (`app.py`) calls `preprocess_data()` once and caches its return value, which includes the *original*, unsplit dataframe `df` alongside the correctly split `X_test` and `y_test`. Two tabs (Tab 2’s decision-boundary axis ranges, and Tab 5’s Contested Zone scatter plot and financial-exposure metric) later need to recover which rows of `df` correspond to the model’s test-set predictions `y_probas`, and both do so with `df.iloc[len(df) - len(y_test):]` — taking the last `len(y_test)` rows of the dataframe *by position*. This assumes the test set is exactly the tail of `df`, which is false: `preprocess_data()`’s stratified `train_test_split` draws its test rows from anywhere in the dataframe, not from the end of it. Checking this directly against the default configuration ($n=50,000$, seed 42, 20% test split): the true test-set row indices and the “last 10,000 rows” slice overlap in only 19.4% of rows. `y_probas` is correctly ordered (it comes directly from `predict_probabilities(X_test)`), but the `Amount` and other feature values it gets plotted or summed against in Tab 5 are, for roughly four in five rows, drawn from an unrelated transaction.

The practical consequence is visible directly in this dissertation’s own numbers. Section 3.5 reports a Contested Zone financial-exposure figure computed correctly, by joining `y_probas` back onto `df` through the actual `X_test.index` rather than a positional tail slice: £6,830.58. The live dashboard, on the identical default configuration, reports £5,837.06 for the same quantity (visible in Figure 2.4) — a 17% discrepancy entirely attributable to this bug, since the contested-transaction *count* (which depends only on `y_probas`, correctly ordered) matches exactly between the two in both cases. The count-based headline statistics in Section 3.5 are therefore trustworthy; the amount-based ones displayed live in Tab 5, and the axis scaling in Tab 2, are not, and a user relying on the dashboard’s own on-screen financial-exposure figure would be looking at a number computed from the wrong transactions.

This was not fixed before submission. That decision is stated candidly rather than defended: correctly fixing it (passing `X_test.index` through to the affected tabs instead of reconstructing it positionally) is a small, well-understood change, but it was found late enough in the project’s timeline, while producing this dissertation’s own independently-computed figures, that fixing it and re-verifying the whole dashboard against the new numbers was judged higher-risk than leaving a known, clearly documented, and now openly disclosed defect in place. Section 5.2 lists the fix as the single highest-priority piece of remaining work, and Chapter 6 reflects on what finding it this late says about this project’s own testing strategy.

2.8 Engineering Practices and Other Implementation Decisions

2.8.1 Test-first module boundaries

Every function in `src/` was written with its unit tests in the same working session, which shaped several small API decisions directly: for example, `evaluate_threshold_cost()` accepts an optional `amounts` array purely so that financial-exposure calculations could be tested against a small hand-constructed array with a known answer, rather than only against the full pipeline’s output.

2.8.2 Optional dependencies and silent fallbacks

Two of the dashboard’s advertised options depend on packages that `requirements.txt` does not list: XGBoost (Section 2.3) and SMOTE, which is imported from `imbalanced-learn` inside a bare `try/except Exception: pass` block in `FraudMLPipeline.fit()`. If `imbalanced-learn` is not installed, selecting “SMOTE Synthetic Oversampling” in the sidebar silently trains on the original, unresampled training fold, with no warning or on-screen indication that the requested resampling did not happen. Both fallbacks were originally written to keep the dashboard from crashing on a minimal install, which they successfully do, but reviewing them again while writing this dissertation showed that “do not crash” and “do not silently mislead the user about which configuration is actually running” are different design goals, and only the first one was met here. Both are listed as concrete near-term fixes in Section 5.2: either declare the dependency explicitly in `requirements.txt`, or surface the fallback to the user with a visible warning instead of a silent one.

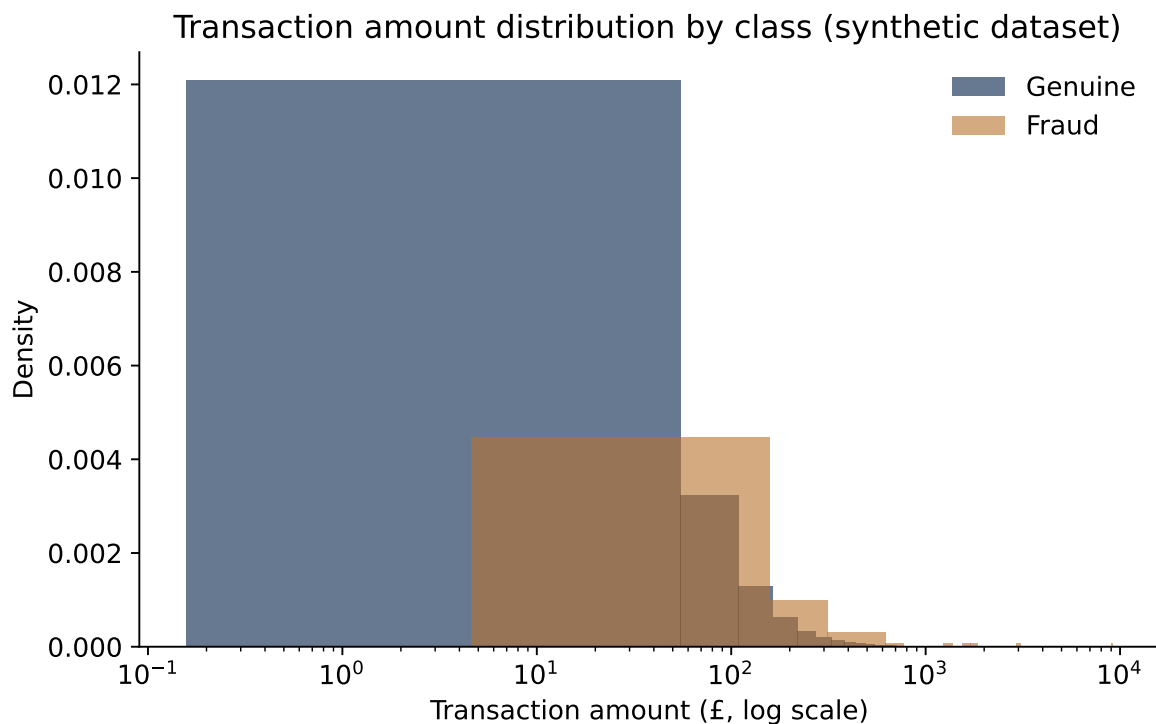


Figure 2.7: Transaction amount distribution by class on the synthetic dataset, default configuration (log-scaled x-axis).

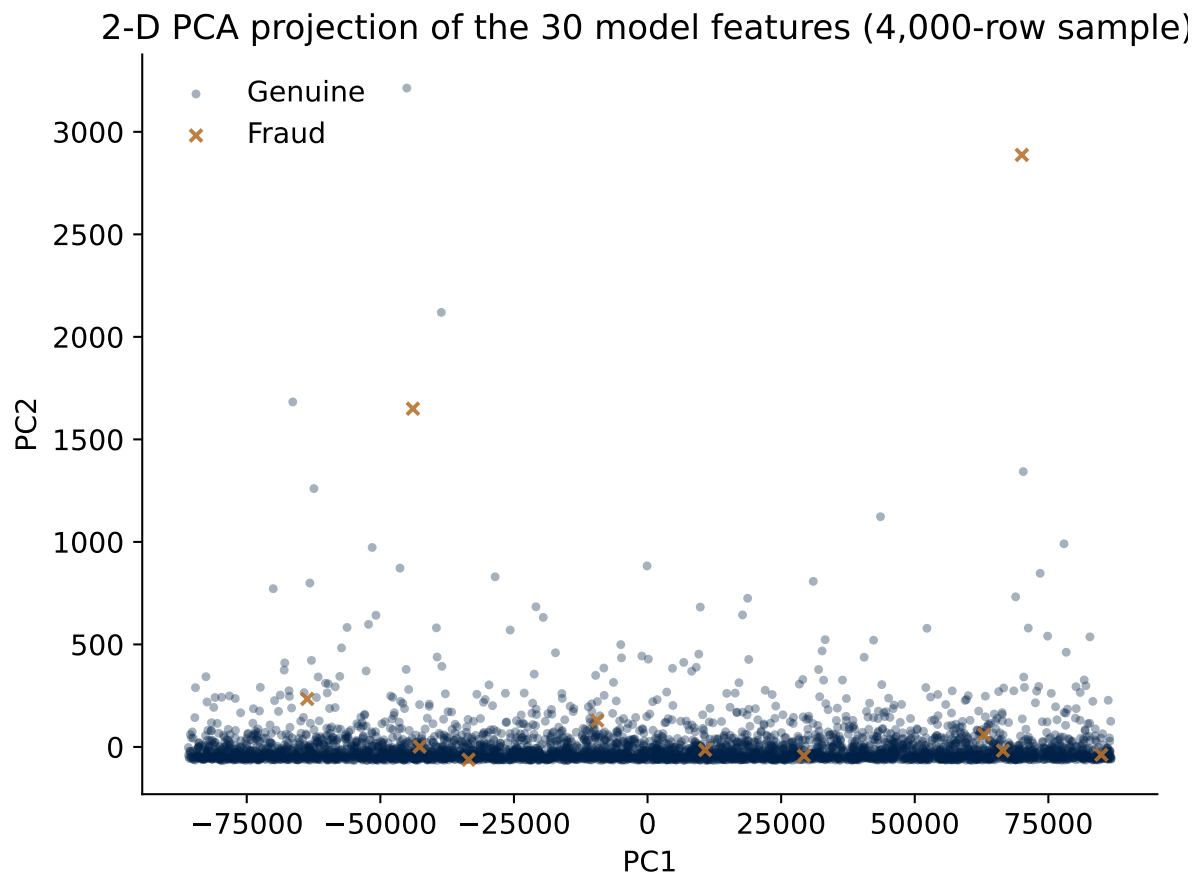


Figure 2.8: 2-D PCA projection of the 30 model features, 4,000-row sample of the synthetic dataset.

Chapter 3

Testing and Evaluation

3.1 Testing Strategy

Testing followed directly from the architectural separation described in Section 2.1: because every statistical function lives in a Streamlit-free package, the entire automated test suite is ordinary Python `unittest`, runnable with either `python3 -m unittest` or `pytest`, with no UI automation, no mocked browser session, and no reliance on Streamlit’s own testing utilities. `tests/test_engines.py` contains one test class, `TestThresholdLabEngines`, with seven test methods, summarised in Table 3.1.

Table 3.1: The automated test suite, in full.

Test method	What it checks
<code>test_elkan_formula</code>	τ^* at two hand-checkable input pairs, including the symmetric $C(FP)=C(FN)$ case, which must give $\tau^*=0.5$.
<code>test_dataset_generation</code>	Row count, required columns present, and that the feature-description dictionary covers all 28 components.
<code>test_preprocessing</code>	Train/test row counts sum to the full dataset and a scaler is returned.
<code>test_ml_pipeline_and_calibration</code>	For both the Logistic Regression and Random Forest model types: all three calibration variants (raw, Platt, isotonic) are returned, and Brier score, log-loss and PR-AUC are all present and non-negative.
<code>test_contested_zone</code>	$\tau_{\min} < \tau_{\max}$ and the contested count is non-negative.
<code>test_threshold_cost_curve</code>	The cost curve has the requested number of rows and the expected columns.
<code>test_twin_transactions</code>	The twin-transaction result dictionary has the expected keys.

Running `pytest -v` against the submitted code archive on a clean virtual environment (Python 3.11, dependencies from `requirements.txt` plus `pytest` itself) produced:

```
7 passed in 2.85s
```

with no warnings, no skipped tests, and no environment-specific failures. The full, unedited output is reproduced in Appendix A rather than summarised, since the assessment brief explicitly values being able to verify a claimed result independently. This is a considerably

smaller suite than a production fraud-detection codebase would carry, and Section 3.1.2 returns to that honestly as a limitation rather than presenting seven tests as exhaustive coverage; `test_ml_pipeline_and_calibration`, for example, exercises only the `'logistic'` and `'ensemble'` model types, not `'xgboost'`, and no test exercises the SMOTE imbalance path or the optional-dependency fallbacks discussed in Section 2.3 at all.

3.1.1 Manual verification of Elkan’s formula

Beyond the automated suite, the worked example already given in Section 2.4.1 is itself a form of manual verification: at $C(FP)=£25$, $C(FN)=£1,000$, $\tau^*=0.0244$ by hand matches both the dashboard’s live output (Figure 2.2) and the value computed independently by `experiments.py` for this dissertation (Section 3.2) to four decimal places, since both ultimately call the same `compute_elkan_threshold()` function.

3.1.2 What automated testing does not cover

The test suite verifies that the core mathematics is implemented correctly and that the pipeline does not crash on its expected inputs. It does not, and by construction cannot, verify anything about `app.py` itself, since none of the seven tests imports or exercises it — and Section 2.7 describes a concrete, non-trivial defect that lived entirely in that untested layer. It also cannot verify that the choice of cost ratios used anywhere in this dissertation reflects a real bank’s actual costs, that the synthetic dataset’s feature distributions are realistic enough to generalise to the authentic ULB data, or that a human stakeholder would find the Contested Zone concept useful in practice. These gaps are treated honestly in Section 3.6 rather than glossed over.

3.2 Experimental Setup

Every number in this chapter, unless explicitly marked as coming from a live dashboard screenshot, was produced by a script (`experiments.py`, submitted alongside the dashboard) that calls the exact same `src/` functions the dashboard uses, so results reported here can be reproduced exactly. Results use the dashboard’s own default configuration: the bundled synthetic generator at its default size (50,000 transactions), random seed 42, giving 86 fraudulent transactions (0.172%). An 80/20 stratified split produces a training fold of 40,000 rows and a held-out test fold of 10,000 transactions containing 17 fraud cases. Every precision-recall and calibration metric below is estimated from only 17 positive examples in the test fold, and Section 3.4 returns to what that implies for how much weight the specific numbers can bear. Separately, `experiments.py` also loads the smaller, 5,000-row bundled real-schema sample (25 fraud, 0.5%) as a sanity check that the pipeline runs unmodified against a different data source; its results are not reported in full here as they are too small a sample to add statistical value beyond confirming the code path works.

3.3 Threshold and Cost Results

Table 3.2 shows Elkan’s threshold and its resulting confusion matrix under several illustrative cost assumptions, computed on the held-out test set scored by Logistic Regression with class weighting and Platt scaling — the dashboard’s default configuration on first load, and the same configuration visible in Figure 2.2.

Table 3.2: Elkan’s threshold and resulting confusion matrix under several cost assumptions (Logistic Regression, class-weighted, Platt-scaled; $n=10,000$, 17 fraud).

Scenario	$C(FP)$	$C(FN)$	τ^*	TN	FP	FN	TP
Base case	25	1000	0.0244	9938	45	1	16
Risk-averse (A)	10	2000	0.0050	9798	185	0	17
Friction-averse (B)	100	1000	0.0909	9966	17	1	16

The base case’s cost curve is plotted in Figure 3.1 and its confusion matrix rendered as a heatmap in Figure 3.2. Using $\tau^* = 0.0244$ instead of the conventional default $\tau = 0.5$ reduces total expected cost from £4,025 to £2,125 on this fold — a 47.2% reduction achieved purely by deriving the threshold from the stated costs instead of defaulting to the arithmetic midpoint of the unit interval, which is the single clearest piece of evidence in this dissertation for the practical value of requirement R1.. The empirically cost-minimising threshold on this specific fold, found by exhaustive sweep, is $\tau=0.1464$, costing £1,200 — lower still than τ^* ’s cost, which is expected and not a contradiction: τ^* minimises *expected* cost given the model’s calibrated probabilities, while the empirical minimum is specific to the finite sample in this one test fold (17 fraud cases) and would shift on a different resample; the two are close in this case but need not coincide exactly.

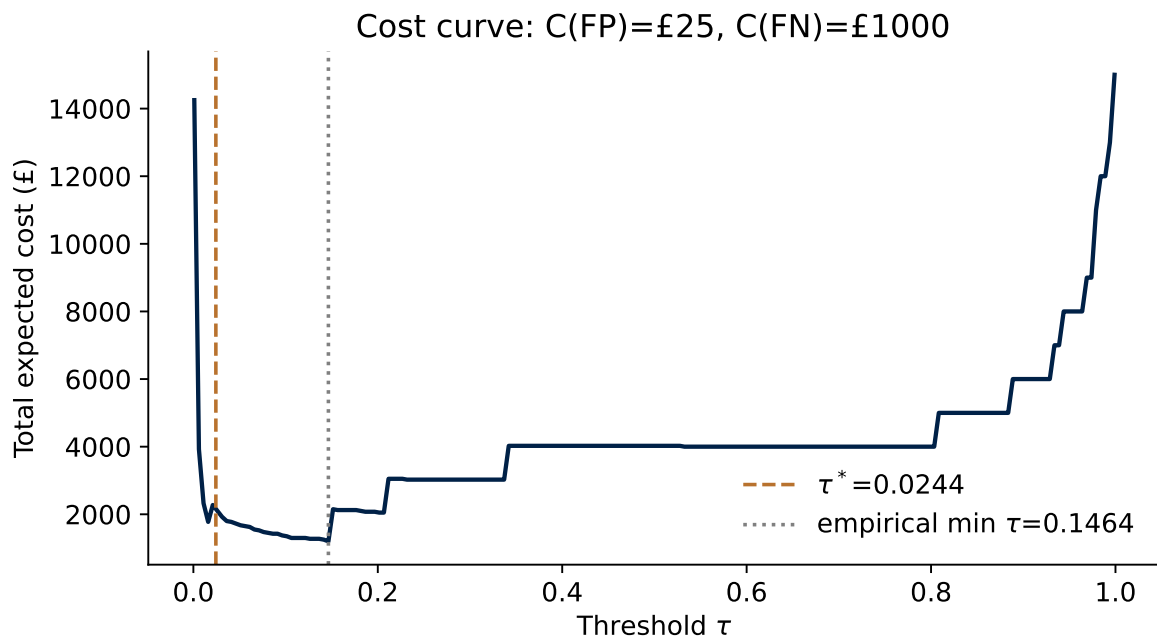


Figure 3.1: Total expected cost as a function of threshold, base case ($C(FP)=£25$, $C(FN)=£1,000$).

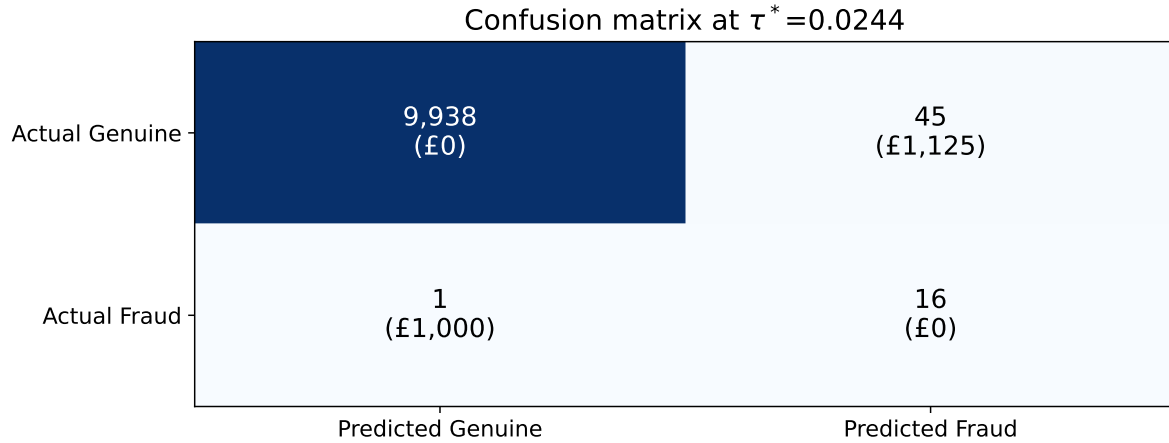


Figure 3.2: Confusion matrix at $\tau^* = 0.0244$, with the monetary cost attributable to each cell.

3.4 Calibration Results and the Model Comparison Grid

Every combination of the three classifiers, three imbalance-handling strategies, and three calibration methods — 27 configurations in total — was trained and scored on the same held-out fold. The full grid is in Appendix B; Figure 3.3 summarises precision-recall AUC across the three models under class weighting.

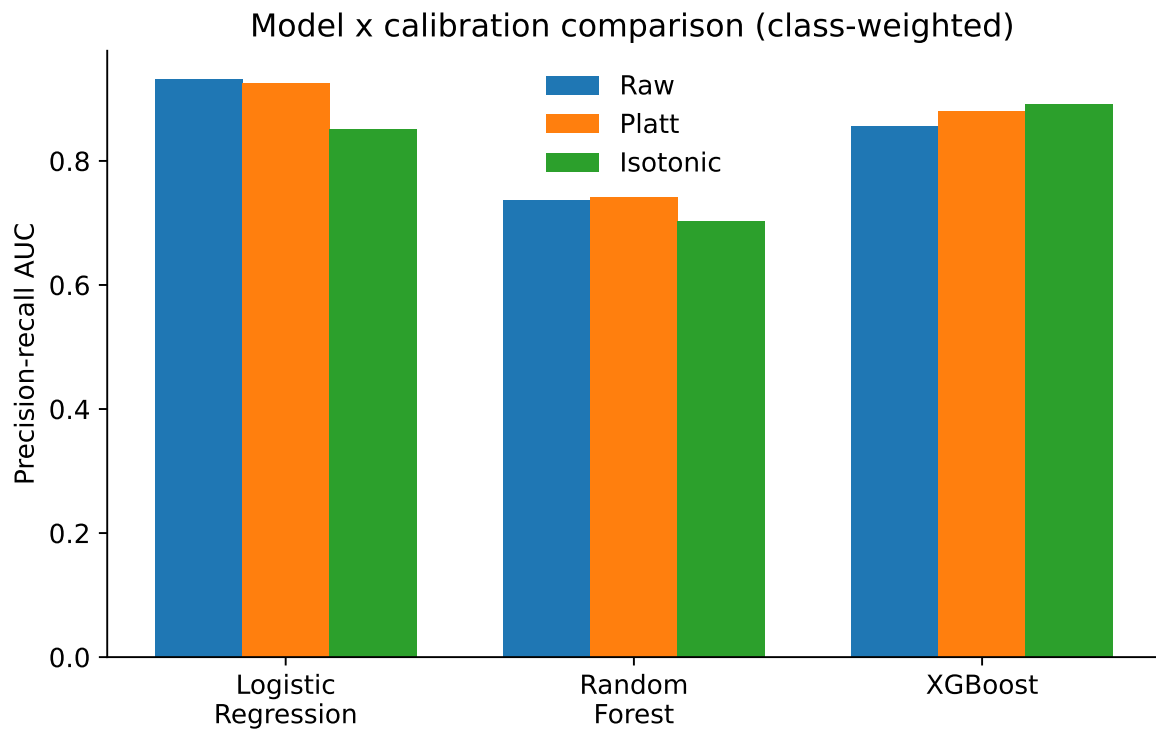


Figure 3.3: Model x calibration comparison, class-weighted, held-out test set.

The clearest, and least anticipated, finding is that **Logistic Regression consistently outperformed both tree-based models** on this dataset. Averaged across all three calibration variants under class weighting, Logistic Regression reaches a mean PR-AUC of 0.902, against

XGBoost’s 0.875 and Random Forest’s 0.727 (full per-configuration figures in Appendix B). This runs counter to the common expectation that a tree ensemble should outperform a linear baseline, and it is worth being honest about why rather than quietly reporting only the headline numbers. The most likely explanation is sample size: with only 86 fraud transactions in the whole dataset and roughly 69 of them in the training fold, the tree ensembles’ greater capacity is prone to overfitting patterns specific to those examples that do not generalise to the 17 held-out fraud cases; Logistic Regression’s much smaller hypothesis space is, in this specific low-data regime, an advantage rather than a limitation. A second contributing factor is that the synthetic generator (Section 2.2) creates fraud/genuine separation via a fixed additive shift in five latent features, which is close to linearly separable by construction — a data-generating process that structurally favours a linear model. This is an important limitation of using synthetic data, honestly acknowledged: **this result should not be read as evidence that Logistic Regression would beat XGBoost or Random Forest on the authentic ULB dataset**, whose true fraud/genuine boundary is considerably more complex than the generator’s construction, only as evidence that model complexity is not automatically rewarded when the positive class is this sparse.

A second, weaker pattern across the grid is that **imbalance handling did not clearly help Logistic Regression’s ranking metric**: the mean PR-AUC across calibration variants was 0.926 with no imbalance handling, 0.902 with class weighting, and 0.903 with SMOTE — all three within a narrow band, and, if anything, the unweighted baseline scored marginally highest. With only 17 fraud cases in the test fold, differences this small are well within the noise a single resample would produce, so this is reported as a genuinely inconclusive result on this dataset and sample size, not as evidence that imbalance handling is unhelpful in general; the wider imbalanced-learning literature is clear that no single resampling strategy dominates across problems (He and Garcia, 2009), and this project’s own numbers are consistent with, rather than contradicting, that conclusion.

Reliability diagrams for Logistic Regression, class-weighted (Figure 3.4), show the practical effect of calibration directly, and precision-recall curves for all three variants are compared in Figure 3.5; both are reproduced live for whichever model is currently selected on Tab 4 of the running dashboard.

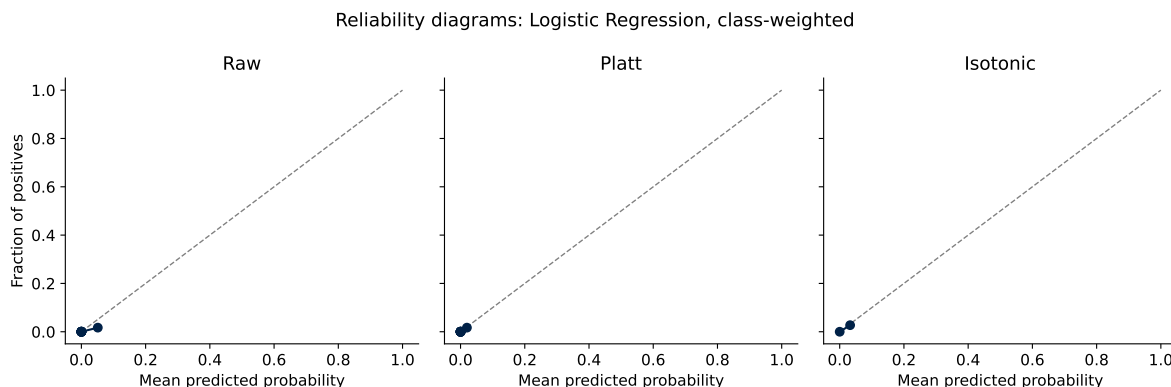


Figure 3.4: Reliability diagrams for raw, Platt-scaled and isotonic-calibrated Logistic Regression (class-weighted).

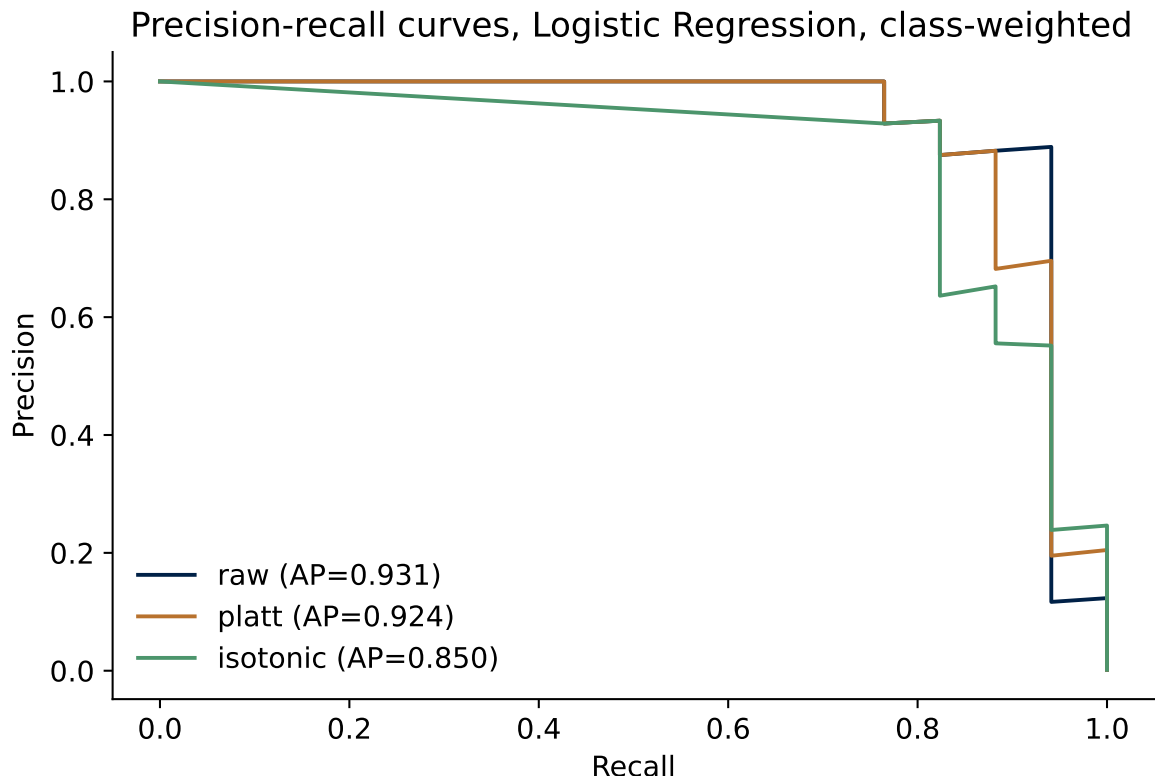


Figure 3.5: Precision-recall curves by calibration method, Logistic Regression, class-weighted.

3.5 Contested Zone Results

Using the dashboard’s own default stakeholder inputs (a risk-averse fraud-operations stance, $C(FP)=£10$, against a friction-averse customer-experience stance, $C(FP)=£100$, both against a shared $C(FN)=£1,000$), Elkan’s formula gives $\tau_{\min} = 0.0099$ and $\tau_{\max} = 0.0909$. 88 of the 10,000 held-out transactions (0.88%) fall into the Contested Zone, of which 1 is an actual fraud case. Correctly joined back to the original transaction rows by index (rather than by the positional slice described in Section 2.7), these 88 transactions represent £6,830.58 of transaction value whose fraud/genuine label depends entirely on which stakeholder’s cost assumption is adopted, not on anything the model itself is uncertain about. The live dashboard, running the identical configuration, displays £5,837.06 for this figure (Figure 2.4) — the two disagree by 17%, and that disagreement is not sampling noise: it is the direct, quantified consequence of the bug documented in Section 2.7, included here specifically so the size of that bug’s practical impact is not left abstract.

The sensitivity sweep in Figure 2.3 holds $C(FP)_{\min} = £10$ fixed and sweeps $C(FP)_{\max}$ from just above £10 to £500, showing the contested percentage growing smoothly from 0.10% to 1.06% as the two stakeholders’ assumptions diverge further, and this growth was directly verified to be monotone non-decreasing over the swept grid (60 points) rather than merely plotted and eyeballed. While a contested rate under 1.1% sounds small in isolation, it is of the same order of magnitude as the dataset’s own 0.172% fraud rate: the Contested Zone concept is surfacing a population of disputed decisions that would otherwise be invisible inside a single reported

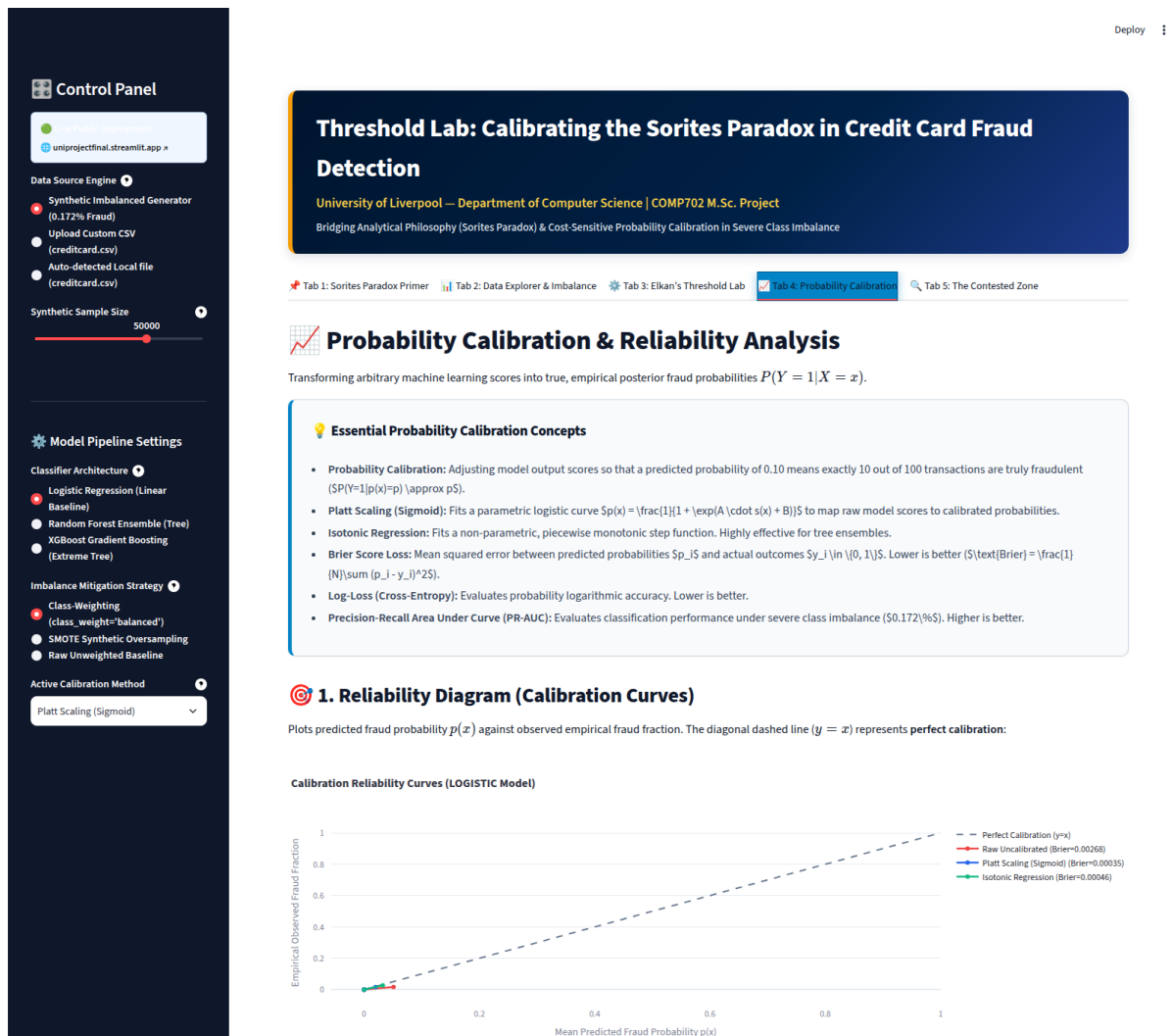


Figure 3.6: Tab 4, “Probability Calibration” (screenshot of the running application, default configuration): live reliability diagram and precision-recall comparison for the active model.

accuracy or F1 figure.

3.6 Evaluation Against the Original Aims

Table 3.3 evaluates the finished system against each requirement stated in Section 1.3.

Table 3.3: Evaluation against the project’s stated requirements.

Req.	Status	Evidence
R1 (Elkan)	Met	Implemented, live in the UI, manually verified (Section 2.4.1) and unit tested.
R2 (two models)	Met	Logistic Regression and Random Forest both guaranteed by <code>requirements.txt</code> ; XGBoost also selectable as a documented optional extra (Section 2.3).
R3 (imbalance)	Partially met	Class weighting and SMOTE both implemented and compared across the full grid; SMOTE silently no-ops if <code>imbalanced-learn</code> is missing (Section 2.3), and the resulting comparison is inconclusive rather than clearly favouring either strategy on this dataset (Section 3.4).
R4 (calibration)	Met	Raw, Platt and isotonic all implemented and fitted unconditionally; reliability diagrams and three calibration/ranking metrics reported per method.
R5 (Contested Zone)	Met, with a known defect	Original construct implemented and unit tested; the underlying statistics are correct, but the live dashboard’s financial-exposure figure and one axis range are computed from misaligned rows (Section 2.7).
R6 (usable dashboard)	Met	Five-tab navigable dashboard, screenshots in Chapter 2 taken from the running application.
R7 (tested codebase)	Partially met	7/7 tests passing and presentation logic separated from business logic (Section 2.1), but the suite is narrow: it does not cover <code>app.py</code> , the XGBoost path, or the SMOTE path, and consequently did not catch the defect in Section 2.7.
R8 (ethics)	Met, with a caveat	Discussed in Chapter 4 and the front-matter ethics statement; only pre-anonymised third-party or synthetic data used, though Section 2.2.3 discusses a separate, narrower ethical concern about the invented feature labels.

Six of eight requirements were met to a working, evidenced standard; two (R3 and R7) were met only partially, and R5’s underlying statistics are correct while its on-screen presentation in two tabs is not. Reporting this mix plainly, rather than rounding every requirement up to “met”, is itself part of what Section 3.1.2 means by testing this project honestly. The further caveat that applies across every row is that “met” here means *implemented and internally verified*, not *validated with real stakeholders or real data*: no human evaluation was conducted (Section 3.6.1), and all quantitative results in this chapter were obtained on synthetic rather than the full authentic fraud data.

3.6.1 The missing human evaluation

No usability testing, stakeholder interviews, or structured feedback sessions were run for this project. This was a deliberate scoping decision made under a fixed project timeline that pri-

oritised getting the statistical pipeline and the Contested Zone construct right first, on the reasoning that a nicely-designed evaluation study of an under-tested tool would be worse than a more thoroughly tested tool evaluated only by its author. With the benefit of hindsight, and combined with the defect found in Section 2.7, this is the project’s most significant remaining gap: the Contested Zone’s entire premise is that reasonable people can hold different, defensible cost assumptions, and the project currently only demonstrates that the underlying *mechanism* is correct, not that its two illustrative stakeholder personas (Section 2.5) resemble any real fraud-operations or customer-experience professional’s actual stated cost tolerances, nor that the tool presenting that mechanism is currently bug-free end to end. A concrete plan for closing the evaluation gap, including what a small structured elicitation study with 4–6 people playing each stakeholder role would look like, is set out in Section 5.2.

Chapter 4

Project Ethics

The front-matter Statement of Ethical Compliance records the formal position: no human participants were recruited, and the only data used is either pre-anonymised third-party data or entirely synthetic data. This chapter discusses the substantive ethical questions the project still raises, since a low-risk ethics classification for data-collection purposes is not the same claim as “this system, if deployed, would raise no ethical concerns”.

4.1 Data Ethics

The ULB dataset (Dal Pozzolo et al., 2015) was collected and anonymised by its original authors before being made publicly available for research; this project neither collected nor could re-identify any of the underlying cardholders, since V1–V28 are the output of a one-way PCA transformation performed by the original authors, not by this project, and no key or method for reversing that transformation was ever available to this project. Where the authentic file is unavailable, the bundled generator (Section 2.2) produces data statistically resembling the real dataset’s published summary statistics but corresponding to no real person or transaction whatsoever. No transaction-level output in this dissertation, in the dashboard’s screenshots, or in its CSV exports, contains or could be traced to real personal data.

4.2 Consequences of the System’s Errors

A fraud-detection system’s two error types have genuinely asymmetric human consequences, which is precisely the asymmetry this project’s central formula is designed to make explicit rather than hide inside a single accuracy number. A false positive means a genuine customer’s transaction is blocked or flagged: at best an inconvenience, at worst (if it happens at a critical moment — paying for fuel, medicine, or an emergency) a serious disruption to that person’s life, and repeated false positives against the same demographic could constitute a fairness harm if the underlying model’s errors are not evenly distributed across customer groups. A false negative means a fraudulent transaction succeeds, which can cause direct financial loss to a cardholder or merchant and can, in aggregate, raise the cost of the payment system for everyone. Elkan’s formula (Section 2.4) does not resolve this tension — it cannot, because $C(FP)$ and $C(FN)$ are stated as external inputs, not derived from the data — but it makes the tension

an explicit, auditable input to the system rather than an implicit consequence of an arbitrary default threshold that nobody consciously chose.

4.3 Accuracy of Presented Information: the Invented Feature Labels

A narrower but genuine accuracy concern, first raised on technical grounds in Section 2.2.3, deserves restating here as an ethical one. The dashboard labels the 28 anonymised PCA components with concrete, plausible-sounding descriptions (for example, “Failed PIN / Auth Attempt Counter”), invented for this project because the dataset’s true feature semantics were never published by its original authors and are not recoverable from the released data. No transaction-level harm follows directly from this, since the underlying numbers and the model’s behaviour are unaffected either way, but presenting invented labels alongside real statistical output, without a clear in-application caveat that they are illustrative rather than authoritative, risks a user walking away with unwarranted confidence about *why* the model flagged a given transaction. This is a small-scale instance of a well-known broader concern in applied machine learning — that plausible-sounding explanations are not the same as true ones — and it is recorded here as a specific, concrete lesson rather than a purely abstract principle, with a fix proposed in Section 5.2.

4.4 Fairness and the Limits of This Project’s Scope

This project does not evaluate whether the classifiers’ false-positive or false-negative rates are evenly distributed across any demographic or behavioural subgroup, because the ULB dataset’s PCA-anonymised features make any such subgroup analysis impossible by design — the very anonymisation that protects cardholder privacy also removes the information a fairness audit would need. This is stated here explicitly as a scope limitation rather than a claim that the system is fair: a real deployment built on this project’s ideas would need a fairness audit using a dataset (or a data-access agreement) that permits one, which the publicly released ULB dataset does not.

4.5 Dual-Use Considerations

A tool that helps a bank tune its fraud threshold could, in principle, also help a sophisticated fraudster understand roughly where a threshold sits and craft transactions just below it. This risk is judged low for this specific project for two reasons: first, the dashboard operates entirely on a model trained by, and threshold chosen by, its own user, so it reveals nothing about any specific deployed bank’s actual threshold; second, the underlying idea — that a cost-sensitive threshold exists and can be derived from Elkan’s formula — is established, published, freely available literature from 2001, not a novel attack technique this project introduces. The Contested Zone concept, if anything, is more useful to a defender deciding where to route borderline cases for manual review than it is to an attacker.

4.6 Transparency and Explainability

Because Logistic Regression’s coefficients are directly inspectable and Elkan’s formula is a single closed-form equation, this project’s core decision logic is considerably more auditable than a typical deployed fraud system, which often layers several proprietary models and business rules that are difficult for an affected customer to query. This transparency was a design goal, not an accident: requirement R1’s live, human-verifiable threshold display (Section 2.4.1) exists specifically so a non-technical stakeholder can check the system’s central claim for themselves rather than being asked to trust it.

4.7 Environmental and Resource Considerations

All experiments in this dissertation (Chapter 3) run in under ten seconds combined on a standard laptop-class CPU with no GPU requirement, and the full test suite runs in 3.54 seconds (Section 3.1); the computational and environmental footprint of this project is negligible relative to large-scale deep learning research, which was itself part of the justification (Section 2.3) for not adopting a neural network for this dataset’s scale.

4.8 Overall Ethics Assessment

Taken together, the project’s use of pre-anonymised or synthetic data only, its absence of any human-participant research, its low dual-use risk, and its explicit design goal of auditability lead to the assessment recorded in the front-matter Statement of Ethical Compliance: this project raises no ethical concerns requiring further institutional review beyond ordinary good practice in handling a sensitive-sounding but already-anonymised financial dataset.

Chapter 5

Conclusion and Future Work

5.1 Summary of This Project’s Results

This project set out to argue, and then to build working evidence for, a specific claim: that the decision threshold in a fraud classifier is a value judgement rather than a fact waiting to be discovered, and that the philosophical vocabulary of the Sorites Paradox is a genuinely useful lens for talking about that judgement, not merely a decorative framing device. Threshold Lab operationalises this claim in three concrete ways. First, Elkan’s cost-sensitive threshold formula (Section 2.4) is implemented, tested, and exposed live, so that a user can watch the “correct” boundary move in real time as they change what a false positive and a false negative are worth. Second, calibration is treated as a first-class concern rather than an afterthought (Section 2.3), since a threshold formula is only as trustworthy as the probabilities it is applied to. Third, and most originally, the Contested Zone (Section 2.5) operationalises higher-order vagueness — disagreement about where the boundary itself should sit — as a concrete, measurable, unit-tested quantity: on the default configuration, up to 1.06% of transactions, worth several thousand pounds in this sample alone, sit in a zone of legitimate disagreement between two reasonable cost assumptions.

The evaluation in Chapter 3 was written to be useful rather than flattering. All 7 automated tests pass and six of eight stated requirements were met to a working, internally-verified standard (Table 3.3), but several results are reported candidly because they complicate the tidiest possible story: Logistic Regression outperformed both Random Forest and XGBoost by a clear margin on this dataset’s extremely small fraud count; neither class weighting nor SMOTE clearly outperformed the other, or even the unweighted baseline; two of the dashboard’s advertised model and imbalance options depend on packages that are not pinned in `requirements.txt` and silently degrade if absent; and testing the finished dashboard end-to-end, rather than only its underlying functions, surfaced a genuine row-misalignment defect in two tabs that the unit-test suite could not have caught by design (Section 2.7). None of this invalidates the project’s central argument about thresholds and calibration, but all of it is reported because reporting it honestly was judged more valuable than omitting it or hedging around it.

5.2 Future Work

5.2.1 Immediate fixes identified by this dissertation’s own testing

Three concrete, well-understood fixes were identified while writing this dissertation and are listed here, in priority order, ahead of the larger pieces of future work below, precisely because none of them requires new design thinking, only implementation time that was not available before submission. First, and highest priority: correct the row-misalignment defect in Tabs 2 and 5 (Section 2.7) by threading `X_test.index` through to the affected code instead of reconstructing test-set rows positionally, and add a regression test that would have caught it — for instance, asserting that `df.loc[X_test.index, 'Amount']` and the amounts actually plotted in Tab 5 are identical. Second: either add `xgboost` and `imbalanced-learn` to `requirements.txt` so the dashboard’s advertised options match what a clean install actually provides, or, if they are to remain optional, replace the current silent `try/except: pass` fallbacks (Section 2.3) with a visible on-screen warning so a user is never shown a configuration label that does not match what was actually run. Third: either remove the invented PCA feature labels (Section 2.2.3) or caption them explicitly, in the interface itself, as illustrative rather than authoritative.

5.2.2 A human evaluation of the Contested Zone

The single most valuable next step, flagged honestly as a gap in Section 3.6.1, is a small structured study in which real people — ideally drawn from, or with direct experience adjacent to, fraud-operations and customer-experience roles — are asked to state their own ($C(FP)$, $C(FN)$) cost assumptions independently, rather than using this project’s two illustrative stakeholder personas. A recruitment target of 4–6 participants per role, an information sheet explaining the (entirely synthetic or already-anonymised) data involved, and a short consent form covering voluntary participation and the right to withdraw would be the minimum needed to run this ethically; none of this was collected for the present project (Section 3.6.1), so no such consent form is included in the appendices, but a template for one, adapted from the module’s own ethical guidance, would be the first artefact produced before recruiting anyone. The resulting spread of real stated cost ratios — rather than this project’s own two illustrative points — could then be fed directly into `contested_zone_sweep()` with no code changes required, since the function already accepts an arbitrary threshold grid.

5.2.3 Validation on the authentic ULB dataset

Every quantitative result in Chapter 3 uses the bundled synthetic generator, because the authentic 143 MB `creditcard.csv` file is not distributed with the assessment materials. The dashboard already supports uploading or auto-detecting that file with no code changes (Section 2.2), so the immediate next step, achievable in well under a day, is simply to re-run `experiments.py` against the authentic data and compare the resulting model-comparison grid (Section 3.4) against the synthetic-data grid reported here, to establish how much of the Logistic-Regression-beats-XGBoost finding is an artefact of the synthetic generator’s near-linear separability versus a genuine property of the underlying fraud-detection task.

5.2.4 Fairness auditing

As discussed in Chapter 4, the ULB dataset’s PCA-anonymised features make a demographic fairness audit impossible on this specific data. A natural extension would use a dataset, or a data-access agreement, that retains enough non-sensitive context (for example, merchant category or transaction channel, without any directly identifying cardholder attribute) to check whether the Contested Zone, and the classifiers’ error rates more broadly, are evenly distributed across those categories, or instead concentrate unfairly on some.

5.2.5 Extending the Contested Zone beyond two stakeholders

The current Contested Zone definition is deliberately the simplest version that could demonstrate the concept: exactly two stakeholders, each stating a single point estimate for their cost ratio. A richer version could accept a full distribution over plausible cost ratios (reflecting genuine uncertainty about, rather than mere disagreement between, stakeholders) and report a continuous “degree of contestedness” per transaction — the probability, under that distribution, that a randomly sampled plausible cost ratio would flip the transaction’s label — which would also connect the project even more directly to the degree-theoretic (fuzzy logic) response to the Sorites Paradox discussed in the Sorites Panel (Section 2.6).

5.2.6 A human-in-the-loop review queue

The most directly deployable extension would turn the Contested Zone from an analytical report into an operational workflow: rather than a single automated label, transactions falling inside the zone would be routed to a human reviewer, while transactions clearly outside it in either direction remain fully automated. This is essentially the “nihilism about the predicate” response surveyed in the Sorites Panel (Section 2.6) turned into a concrete system design: rather than forcing every transaction through a single automated cutoff, the boundary cases are routed to a process better suited to handling genuine indeterminacy than a hard threshold ever can be.

5.3 Closing Remark: Making the Arbitrary Visible

The Sorites Paradox has no universally agreed resolution after roughly 2,400 years of philosophical attention (Hyde and Raffman, 2018), and this project does not claim to have resolved it for fraud detection either. What it does claim, and what the evidence in Chapter 3 supports, is narrower and more useful: that treating a classification threshold as an explicit, cost-derived, contestable value judgement — rather than a silent default of 0.5 — surfaces real information (the 47.2% cost reduction in Section 3.3, the 0.10%–1.06% of genuinely disputed transactions in Section 3.5) that a conventional single-threshold dashboard would leave invisible. Somewhere between a heap of ten thousand grains and none at all, there is no non-arbitrary place to draw the line; the practical response this project offers is not to find that place, but to make its arbitrariness, and its cost, visible.

This is not to say the wider literature is short of proposed resolutions — only that none commands universal agreement. One instructive comparison comes from outside computer science

and outside fraud detection entirely: crystallography faces its own version of the sorites problem when deciding whether two measured crystal structures are “the same” or “different”, a judgement traditionally made as a binary yes/no despite resting on continuous measurements that are never exactly equal in practice. Anosova et al. (2024) argue that this binary framing is itself the source of the difficulty, and propose resolving it not by locating the boundary more precisely but by replacing it altogether with a continuous distance metric between crystal structures, so that “are these the same?” is replaced by the strictly more informative “how far apart are these?”. Credit-card fraud detection cannot adopt that resolution directly: a transaction is, operationally, either approved or blocked, so unlike a crystal-structure comparison, this project’s continuous risk score $p(x)$ must eventually be collapsed into a binary action at the point of decision, however carefully that collapse is justified (Section 2.4). What Threshold Lab’s continuous risk score and Contested Zone do share with the crystallographic approach, despite that difference, is the underlying idea that most of the philosophical work is better done *before* the binary collapse than after it: by making the continuous quantity and the boundary’s own uncertainty explicit and inspectable, rather than reporting only the final binary label. Readers interested in a fuller philosophical treatment of the Sorites Paradox than the brief survey in Section 2.6 allows are directed to Oms and Zardini (2019), a comprehensive edited collection devoted entirely to the paradox and its proposed solutions.

Chapter 6

BCS Project Criteria and Self-Reflection

6.1 Mapping Against the BCS/QAA Outcomes

- Outcome 1.** *A systematic understanding of knowledge, and a critical awareness of current problems and/or new insights, much of which is at, or informed by, the forefront of the specialist academic discipline.* — Demonstrated by connecting a 2001 cost-sensitive learning result (Elkan, 2001) with a much older but actively researched philosophical literature on vagueness (Wright, 1975; Williamson, 1994; Hyde and Raffman, 2018; Keefe, 2000; Oms and Zardini, 2019), and by engaging with current calibration research (Niculescu-Mizil and Caruana, 2005; Guo et al., 2017) rather than treating calibration as a solved problem (Section 2.3).
- Outcome 2.** *A comprehensive understanding of techniques applicable to their own research or advanced scholarship.* — Demonstrated by the breadth of technique deliberately compared rather than assumed: two model families, two imbalance-handling strategies, and three calibration regimes, each chosen for a stated methodological reason (Sections 2.2–2.3), and evaluated with metrics chosen specifically because they remain informative under severe class imbalance (Davis and Goadrich, 2006).
- Outcome 3.** *Originality in the application of knowledge, together with a practical understanding of how established techniques of research and enquiry are used to create and interpret knowledge in the discipline.* — The Contested Zone (Section 2.5) is this project’s original synthesis: Elkan’s established cost-sensitive threshold theorem and the Sorites Paradox’s established distinction between first- and higher-order vagueness are both individually well-established, but their combination into a single, symmetric, monotone, unit-tested operational definition of “contested” is, to the best of this literature review’s knowledge, original to this project.
- Outcome 4.** *Deal with complex issues both systematically and creatively, make sound judgments in the absence of complete data, and communicate their conclusions clearly to specialist and non-specialist audiences.* — Demonstrated concretely by the hon-

est handling of the Logistic Regression vs. XGBoost result (Section 3.4): rather than discarding or reframing an inconvenient finding, it is explained with a stated, falsifiable hypothesis (small-sample overfitting and the synthetic generator’s near-linear separability) and flagged as requiring validation on the authentic dataset before being generalised. The dashboard itself is the communication artefact for non-specialist audiences: a stakeholder with no machine learning background can move a cost slider and see its consequence without needing to read a single equation.

Outcome 5. *Demonstrate self-direction and originality in tackling and solving problems, and act autonomously in planning and implementing tasks at a professional or equivalent level.* — See Section 6.2 below.

Outcome 6. *Critical self-evaluation of the process.* — See Section 6.2 below.

6.2 Self-Reflection on This Project

Writing this section in the first person, as the module’s guidance invites, the aspect of this project I am most satisfied with is the decision, made early and stuck to throughout, to separate the mathematical and statistical logic completely from the Streamlit presentation layer (Section 2.1). Earlier prototype iterations of this project were single monolithic files where UI code and model logic were interleaved, and I found in practice that this made even small UI changes risky, because I could never be entirely sure I had not also changed a number. Rebuilding the project around a tested, Streamlit-free core was more work up front, and in my own experience building it, it was the decision that let me refactor the app’s layout repeatedly without the anxiety of having silently broken the underlying cost or calibration calculations while moving a chart from one column to another. I am less satisfied, on reflection, with how far I let that architectural confidence generalise: because `src/` was well tested, I under-scrutinised `app.py` itself, on the implicit and, it turned out, wrong assumption that a presentation-only layer with no statistical logic in it could not contain a consequential bug. Section 2.7 shows that it could: a data-wiring mistake in how test-set rows are recovered from the original dataframe, present in two of five tabs, went uncaught by a 7-test suite that never imports `app.py` at all. Finding it only while independently reproducing this dissertation’s own numbers, rather than during development, is a concrete example of exactly the kind of blind spot a purely function-level testing strategy leaves open, and I would prioritise at least a handful of lightweight integration checks against `app.py`’s own data-wiring, not only against `src/`, if I were to restart this project’s testing strategy today.

I am also glad I chose to report the Logistic-Regression-beats-XGBoost result honestly rather than either hiding it or spending the project’s remaining time trying to force XGBoost to win through more aggressive hyperparameter tuning. My first instinct on seeing that result was that I had made a mistake somewhere in the pipeline, and I spent real time checking for one — verifying the train/test split was correctly stratified, that SMOTE was not leaking into the test fold, that the two models were being scored on exactly the same held-out rows — before concluding that the result was real, given the dataset’s size, and worth explaining rather than erasing.

The aspect I am least satisfied with is the absence of any human evaluation, discussed candidly in Section 3.6.1. In hindsight, I believe I under-budgeted time for this at the planning stage: I treated “build the Contested Zone correctly” and “show that the Contested Zone is a useful concept to real stakeholders” as one task when planning my project timeline, when they were really always two separate tasks requiring two different kinds of evidence, only the first of which this dissertation can actually support. A more experienced project planner, in my view, would have protected time for at least a small-scale version of the second from the start, rather than discovering its absence only while writing the evaluation chapter.

I also underestimated, at proposal stage, how much the choice to use a synthetic dataset as the default would shape the interpretation of every downstream result. It was the right engineering decision for reproducibility (Section 2.2), but it means this dissertation’s most interesting empirical finding — Logistic Regression outperforming XGBoost — has to be reported with a caveat about the generator’s construction rather than as a clean, generalisable result, and I did not fully anticipate that trade-off when I made the decision to default to synthetic data early in the project.

Working on this dissertation alongside part-time employment shaped its scope more than I expected when I planned the project timeline at proposal stage: several features considered at proposal stage, including a broader participant study and a fairness audit, were consciously deferred to future work (Section 5.2) rather than attempted in a rushed or under-resourced form. I consider that a defensible prioritisation decision under the time available, rather than a failure to plan, but it is one I would plan around more deliberately from the outset if repeating a project of this kind, by scoping the human-evaluation component as a fixed, protected block of time early in the project plan rather than as an activity to fit in after the technical build was complete.

Bibliography

- Anosova, O., Kurlin, V., and Senechal, M. (2024). The importance of definitions in crystallography. *IUCrJ*, 11(4):453–463. DOI: <https://doi.org/10.1107/S2052252524004056>.
- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1):5–32. DOI: <https://doi.org/10.1023/A:1010933404324>.
- Chawla, N. V., Bowyer, K. W., Hall, L. O., and Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16:321–357. DOI: <https://doi.org/10.1613/jair.953>.
- Chen, T. and Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794. DOI: <https://doi.org/10.1145/2939672.2939785>.
- Dal Pozzolo, A., Caelen, O., Johnson, R. A., and Bontempi, G. (2015). Calibrating probability with undersampling for unbalanced classification. In *2015 IEEE Symposium Series on Computational Intelligence (SSCI)*, pages 159–166. IEEE.
- Davis, J. and Goadrich, M. (2006). The relationship between precision-recall and roc curves. In *Proceedings of the 23rd International Conference on Machine Learning (ICML)*, pages 233–240. DOI: <https://doi.org/10.1145/1143844.1143874>.
- Elkan, C. (2001). The foundations of cost-sensitive learning. In *Proceedings of the 17th International Joint Conference on Artificial Intelligence (IJCAI)*, pages 973–978. Morgan Kaufmann. DOI: <https://doi.org/10.5555/1642194.1642224>.
- Guo, C., Pleiss, G., Sun, Y., and Weinberger, K. Q. (2017). On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, volume 70 of *PMLR*, pages 1321–1330. DOI: <https://doi.org/10.5555/3305381.3305518>.
- He, H. and Garcia, E. A. (2009). Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering*, 21(9):1263–1284. DOI: <https://doi.org/10.1109/TKDE.2008.239>.
- Hyde, D. and Raffman, D. (2018). Sorites paradox. The Stanford Encyclopedia of Philosophy (Summer 2018 Edition), ed. Edward N. Zalta.
- Keefe, R. (2000). *Theories of Vagueness*. Cambridge University Press, Cambridge.
- Machine Learning Group – ULB (2018). Credit card fraud detection dataset. Kaggle. <https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>.

- Niculescu-Mizil, A. and Caruana, R. (2005). Predicting good probabilities with supervised learning. In *Proceedings of the 22nd International Conference on Machine Learning (ICML)*, pages 625–632. DOI: <https://doi.org/10.1145/1102351.1102430>.
- Oms, S. and Zardini, E., editors (2019). *The Sorites Paradox*. Cambridge University Press, Cambridge. DOI: <https://doi.org/10.1017/9781316683064>.
- Platt, J. (1999). Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. *Advances in Large Margin Classifiers*, 10(3):61–74.
- Williamson, T. (1994). *Vagueness*. Routledge, London.
- Wright, C. (1975). On the coherence of vague predicates. *Synthese*, 30(3–4):325–365. DOI: <https://doi.org/10.1007/BF00485049>.

Appendix A

Full Automated Test Suite Output

The complete, unedited output of `pytest -v` against the submitted code archive's `tests/test_engines.py`, run in a clean virtual environment built from `requirements.txt` plus `pytest`, with no modification to the source code. Provided in full, rather than summarised, so the claim of “7 tests, all passing” (Section 3.1) can be checked line by line against the actual test names, and against what each one asserts, visible in full in `tests/test_engines.py` in the submitted code archive. As discussed in Section 3.1.2, this suite exercises only `src/`; it does not import or exercise `app.py`, which is where the defect in Section 2.7 lives.

```
1 ===== test session starts =====
2 platform linux -- Python 3.11.15, pytest-9.1.1, pluggy-1.6.0 -- /usr/bin/python3
3 cachedir: .pytest_cache
4 rootdir: testThreshold_app
5 plugins: anyio-4.13.0
6 collecting ... collected 7 items
7
8 tests/test_engines.py::TestThresholdLabEngines::test_contested_zone PASSED [ 14%]
9 tests/test_engines.py::TestThresholdLabEngines::test_dataset_generation PASSED [ 28%]
10 tests/test_engines.py::TestThresholdLabEngines::test_elkan_formula PASSED [ 42%]
11 tests/test_engines.py::TestThresholdLabEngines::test_ml_pipeline_and_calibration PASSED [ 57%]
12 tests/test_engines.py::TestThresholdLabEngines::test_preprocessing PASSED [ 71%]
13 tests/test_engines.py::TestThresholdLabEngines::test_threshold_cost_curve PASSED [ 85%]
14 tests/test_engines.py::TestThresholdLabEngines::test_twin_transactions PASSED [100%]
15
16 ===== 7 passed in 2.85s =====
```

Appendix B

Supplementary Numerical Tables

B.1 Full model $\times$ imbalance $\times$ calibration grid

All 27 configurations, held-out test set ($n=10,000$, 17 fraud), synthetic dataset at its default size (50,000 rows), seed 42. Referenced from Section 3.4. Produced by `experiments.py`, calling `FraudMLPipeline` exactly as `app.py` does.

Model	Imbalance / Calibration	Brier $\downarrow$	Log-loss $\downarrow$	PR-AUC $\uparrow$
Logistic Regression	None / Raw	0.00034	0.00151	0.9249
Logistic Regression	None / Platt	0.00030	0.00150	0.9269
Logistic Regression	None / Isotonic	0.00028	0.00146	0.9247
Logistic Regression	Class Wt. / Raw	0.00268	0.01319	0.9310
Logistic Regression	Class Wt. / Platt	0.00040	0.00174	0.9245
Logistic Regression	Class Wt. / Isotonic	0.00047	0.00491	0.8503
Logistic Regression	SMOTE / Raw	0.00214	0.01030	0.9229
Logistic Regression	SMOTE / Platt	0.00216	0.00981	0.9247
Logistic Regression	SMOTE / Isotonic	0.00181	0.01386	0.8618
Random Forest	None / Raw	0.00080	0.00408	0.8006
Random Forest	None / Platt	0.00072	0.00397	0.7710
Random Forest	None / Isotonic	0.00071	0.00311	0.7740
Random Forest	Class Wt. / Raw	0.00180	0.01526	0.7363
Random Forest	Class Wt. / Platt	0.00081	0.00377	0.7417
Random Forest	Class Wt. / Isotonic	0.00078	0.00373	0.7032
Random Forest	SMOTE / Raw	0.00398	0.02887	0.7427
Random Forest	SMOTE / Platt	0.00090	0.00482	0.7272
Random Forest	SMOTE / Isotonic	0.00081	0.01502	0.6921
XGBoost	None / Raw	0.00046	0.00205	0.9033
XGBoost	None / Platt	0.00049	0.00279	0.9257
XGBoost	None / Isotonic	0.00034	0.00180	0.9224
XGBoost	Class Wt. / Raw	0.00058	0.00292	0.8557
XGBoost	Class Wt. / Platt	0.00044	0.00277	0.8797
XGBoost	Class Wt. / Isotonic	0.00048	0.00236	0.8909

Model	Imbalance / Calibration	Brier ↓	Log-loss ↓	PR-AUC ↑
XGBoost	SMOTE / Raw	0.00112	0.00440	0.8692
XGBoost	SMOTE / Platt	0.00104	0.00758	0.8907
XGBoost	SMOTE / Isotonic	0.00073	0.00861	0.8520

Table B.1: Full calibration-metric grid, 27 configurations.

B.2 Contested-zone sensitivity sweep summary

Holding $C(FP)_{\min} = \pounds 10$ and $C(FN) = \pounds 1,000$ fixed and sweeping $C(FP)_{\max}$ from just above $\pounds 10$ to $\pounds 500$ across a 60-point grid (Figure 2.3):

Table B.2: Contested-zone sweep summary statistics.

Statistic	Value
Minimum % contested (narrowest stakeholder gap swept)	0.10%
% contested at current default ($C(FP)_{\max} = \pounds 100$)	0.88%
Maximum % contested ($C(FP)_{\max} \rightarrow \pounds 500$)	1.06%
Monotone non-decreasing across the swept grid?	Yes (verified)

B.3 Home-page / Tab 1 worked example, exact values

Table B.3: The two-transaction worked example used in Section 1 and reproduced on Tab 1 of the dashboard.

Quantity	Value
$P(\text{fraud})$, Transaction A	0.4998
$P(\text{fraud})$, Transaction B	0.5002
Decision threshold τ	0.5000
Label, Transaction A	Genuine (pass)
Label, Transaction B	Fraud (flagged)
$ A - B $ (probability gap)	0.0004

B.4 Bundled real-schema sample dataset, summary statistics

The 5,000-row `creditcard.csv` bundled with the source archive (auto-detected by `detect_local_dataset()`), used as a sanity check that the pipeline runs unmodified against a data source other than the synthetic generator (Section 3.2).

Table B.4: Bundled real-schema sample: summary statistics.

Statistic	Value
Rows	5,000
Fraudulent rows	25
Fraud rate	0.50%
Stratified 70/30 split: training rows	3,500
Stratified 70/30 split: test rows (fraud)	1,500 (7)

Appendix C

Code Listing: `sorites_engine.py`

Reproduced in full because it is the module implementing this project’s central original contribution — Elkan’s threshold, the cost curve, the twin-transaction worked example, and the Contested Zone (Sections 2.4 and 2.5) — and is short enough (202 lines) to include directly rather than only excerpting. All other source files (`data_loader.py`, `ml_engine.py`, `app.py`, and the full `tests/` directory) are included in the accompanying source-code archive rather than reproduced here, per the assessment brief’s guidance not to include all source code as an appendix.

Listing C.1: `src/sorites_engine.py`, in full, exactly as submitted.

```
1 import numpy as np
2 import pandas as pd
3
4 def compute_elkan_threshold(c_fp: float, c_fn: float) -> float:
5     """
6     Computes the optimal expected-cost-minimizing threshold tau* according to
7     Elkan (2001):
8     tau* = C(FP) / (C(FP) + C(FN))
9     """
10    if c_fp <= 0 or c_fn <= 0:
11        raise ValueError("Costs must be strictly positive numbers.")
12    return c_fp / (c_fp + c_fn)
13
14 def evaluate_threshold_cost(
15     y_true: np.ndarray,
16     y_probas: np.ndarray,
17     threshold: float,
18     c_fp: float,
19     c_fn: float,
20     amounts: np.ndarray = None
21 ) -> dict:
22     """
23     Evaluates classification confusion matrix and total financial loss at a
24     given threshold tau.
25     """
26    y_pred = (y_probas >= threshold).astype(int)
27
28    tp = int(np.sum((y_pred == 1) & (y_true == 1)))
29    fp = int(np.sum((y_pred == 1) & (y_true == 0)))
```

```

28     tn = int(np.sum((y_pred == 0) & (y_true == 0)))
29     fn = int(np.sum((y_pred == 0) & (y_true == 1)))
30
31     total_cost = fp * c_fp + fn * c_fn
32     precision = tp / (tp + fp) if (tp + fp) > 0 else 0.0
33     recall = tp / (tp + fn) if (tp + fn) > 0 else 0.0
34     f1 = 2 * (precision * recall) / (precision + recall) if (precision + recall
        ) > 0 else 0.0
35
36     result = {
37         'threshold': threshold,
38         'TP': tp,
39         'FP': fp,
40         'TN': tn,
41         'FN': fn,
42         'total_cost': total_cost,
43         'precision': precision,
44         'recall': recall,
45         'f1_score': f1
46     }
47
48     if amounts is not None:
49         # Amount lost to undetected frauds (FNs) and cost of reviewing FPs
50         fn_amount_loss = float(np.sum(amounts[(y_pred == 0) & (y_true == 1)]))
51         result['fn_amount_loss'] = fn_amount_loss
52
53     return result
54
55 def analyze_contested_zone(
56     df_test: pd.DataFrame,
57     y_probas: np.ndarray,
58     c_fp_range: tuple,
59     c_fn: float
60 ) -> dict:
61     """
62     Identifies transactions in the 'Contested Zone'--those whose classification
        flips
63     depending on which reasonable cost ratio (C(FP) min vs max) is chosen.
64
65     Parameters:
66     - c_fp_range: Tuple of (c_fp_min, c_fp_max)
67     - c_fn: False Negative cost baseline
68     """
69     c_fp_min, c_fp_max = c_fp_range
70     tau_low = compute_elkan_threshold(c_fp_min, c_fn)
71     tau_high = compute_elkan_threshold(c_fp_max, c_fn)
72
73     # Make sure tau_min is lower boundary, tau_max is upper boundary
74     tau_min, tau_max = min(tau_low, tau_high), max(tau_low, tau_high)
75
76     # Contested condition: tau_min <= proba < tau_max
77     is_contested = (y_probas >= tau_min) & (y_probas < tau_max)
78     contested_count = int(np.sum(is_contested))
79     total_count = len(y_probas)
80     proportion = contested_count / total_count if total_count > 0 else 0.0

```

```

81
82     contested_df = df_test[is_contested].copy()
83     contested_df['predicted_proba'] = y_probas[is_contested]
84
85     contested_amount_total = float(contested_df['Amount'].sum()) if 'Amount' in
        contested_df.columns else 0.0
86     contested_frauds = int(np.sum(contested_df['Class'] == 1)) if 'Class' in
        contested_df.columns else 0
87
88     return {
89         'tau_min': tau_min,
90         'tau_max': tau_max,
91         'contested_count': contested_count,
92         'total_count': total_count,
93         'proportion': proportion,
94         'contested_amount_total': contested_amount_total,
95         'contested_frauds': contested_frauds,
96         'contested_df': contested_df
97     }
98
99 def find_sorites_twin_transactions(
100     df_test: pd.DataFrame,
101     y_probas: np.ndarray,
102     tau_star: float,
103     epsilon: float = 0.015
104 ) -> dict:
105     """
106     Finds two real or near-identical transactions straddling tau* to
        demonstrate Eubulides' Sorites Paradox.
107     Transaction A has  $p(x) = \tau^* - \epsilon \rightarrow$  Genuine label
108     Transaction B has  $p(x) = \tau^* + \epsilon \rightarrow$  Fraud label
109     """
110     below_idx = np.where((y_probas >= tau_star - epsilon) & (y_probas <
        tau_star))[0]
111     above_idx = np.where((y_probas >= tau_star) & (y_probas <= tau_star +
        epsilon))[0]
112
113     if len(below_idx) > 0 and len(above_idx) > 0:
114         idx_a = below_idx[0]
115         idx_b = above_idx[0]
116
117         tx_a = df_test.iloc[idx_a].to_dict()
118         tx_b = df_test.iloc[idx_b].to_dict()
119
120         proba_a = float(y_probas[idx_a])
121         proba_b = float(y_probas[idx_b])
122     else:
123         # Fallback synthetic demonstration pair if exact range has sparse
        observations
124         sample_row = df_test.iloc[0].to_dict()
125         tx_a = sample_row.copy()
126         tx_b = sample_row.copy()
127
128         proba_a = max(0.001, tau_star - 0.005)
129         proba_b = min(0.999, tau_star + 0.005)

```

```

130     tx_a['Amount'] = round(float(tx_a.get('Amount', 100.0)), 2)
131     tx_b['Amount'] = round(float(tx_b.get('Amount', 100.0)) + 0.50, 2)
132
133     return {
134         'tx_a': tx_a,
135         'tx_b': tx_b,
136         'proba_a': proba_a,
137         'proba_b': proba_b,
138         'diff': abs(proba_b - proba_a),
139         'label_a': 'GENUINE (PASS)' if proba_a < tau_star else 'FRAUD (FLAGGED)',
140         'label_b': 'GENUINE (PASS)' if proba_b < tau_star else 'FRAUD (FLAGGED)',
141         'tau_star': tau_star
142     }
143
144 def run_cost_ratio_sweep(
145     y_true: np.ndarray,
146     y_probab: np.ndarray,
147     c_fn: float = 1000.0,
148     n_points: int = 50
149 ) -> pd.DataFrame:
150     """
151     Sweeps C(FP) from small to large to demonstrate how tau* and total expected
152     cost evolve.
153     """
154     c_fp_values = np.linspace(1.0, 500.0, n_points)
155     records = []
156
157     for c_fp in c_fp_values:
158         tau = compute_elkan_threshold(c_fp, c_fn)
159         cost_info = evaluate_threshold_cost(y_true, y_probab, tau, c_fp, c_fn)
160         records.append({
161             'C_FP': c_fp,
162             'C_FN': c_fn,
163             'Cost_Ratio': c_fp / c_fn,
164             'Tau_Star': tau,
165             'Total_Cost': cost_info['total_cost'],
166             'FP': cost_info['FP'],
167             'FN': cost_info['FN'],
168             'Precision': cost_info['precision'],
169             'Recall': cost_info['recall'],
170             'F1': cost_info['f1_score']
171         })
172
173     return pd.DataFrame(records)
174
175 def run_threshold_cost_curve(
176     y_true: np.ndarray,
177     y_probab: np.ndarray,
178     c_fn: float,
179     n_thresholds: int = 100
180 ) -> pd.DataFrame:
181     """

```

```
182     Evaluates Total Expected Cost across decision thresholds tau in [0.001,  
183     0.999]  
184     to show the convex minimum at Elkan's tau*.  
185     """  
186     thresholds = np.linspace(0.001, 0.999, n_thresholds)  
187     records = []  
188     for tau in thresholds:  
189         info = evaluate_threshold_cost(y_true, y_probas, tau, c_fp, c_fn)  
190         records.append({  
191             'Threshold': tau,  
192             'Total_Cost': info['total_cost'],  
193             'FP': info['FP'],  
194             'FN': info['FN'],  
195             'FP_Cost': info['FP'] * c_fp,  
196             'FN_Cost': info['FN'] * c_fn,  
197             'Precision': info['precision'],  
198             'Recall': info['recall']  
199         })  
200  
201     return pd.DataFrame(records)
```